MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA CARTOGRÁFICA

VINICIUS HENEQUIM CORRÊA
DANIEL SANTANA

BUSCA DE GEOINFORMAÇÃO EM CATÁLOGO DE DADOS ESPACIAIS UTILIZANDO MODELOS DE LINGUAGEM DE GRANDE PORTE COM TOOL CALLING

RIO DE JANEIRO
2026

©2026
INSTITUTO MILITAR DE ENGENHARIA
Praça General Tibúrcio, 80 – Praia Vermelha
Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmar ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

Corrêa, Vinicius Henequim; Santana, Daniel.
Busca de Geoinformação em Catálogo de Dados Espaciais Utilizando Modelos de Linguagem de Grande Porte com Tool Calling / Vinicius Henequim Corrêa e Daniel Santana. – Rio de Janeiro, 2026.
87 f.

Orientador(es): Ivanildo Barbosa.

Projeto Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia Cartográfica, 2026.

1. Modelos amplos de linguagem. 2. *Tool calling*. 3. Catálogo de dados espaciais. 4. Disseminação da informação geoespacial. 5. Cartografia sistemática. 6. Ollama. 7. Avaliação de modelos. i. Barbosa, Ivanildo (orient.) ii. Título

VINICIUS HENEQUIM CORRÊA
DANIEL SANTANA

BUSCA DE GEOINFORMAÇÃO EM CATÁLOGO DE DADOS ESPACIAIS UTILIZANDO MODELOS DE LINGUAGEM DE GRANDE PORTE COM TOOL CALLING

Projeto Final de Curso apresentado ao Curso de Graduação em Engenharia Cartográfica do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia Cartográfica.

Orientador(es): Ivanildo Barbosa, DSc

Rio de Janeiro

2026

A Cristo Rei.

RESUMO

O acervo cartográfico da Diretoria de Serviço Geográfico (DSG), disponível no Banco de Dados Geográficos do Exército (BDGEx), é organizado por nomenclaturas técnicas, como o Índice de Nomenclatura e o Mapa Índice, que tendem a restringir o seu uso por não especialistas. Para disseminar a informação geoespacial, deseja-se a busca em linguagem natural, traduzida por um modelo de linguagem nos critérios do catálogo. O projeto `prototipo_busca_llm`, desenvolvido pelo Maj Diniz no 1º Centro de Geoinformação (1º CGEO), fazia essa tradução com saída estruturada, em que o modelo preenche um formato fixo, com um só modelo local (Phi-4 14B) e um dicionário de normalização, sem protocolo de avaliação. O objetivo deste trabalho foi adequar esse projeto ao *tool calling*, em que o modelo escolhe se chama ferramentas e com quais argumentos. A primeira das três versões desenvolvidas oferece só a ferramenta de busca, numa única chamada ao modelo. A segunda acrescenta uma ferramenta de recusa e escreve nas descrições das ferramentas convenções do manual que define o gabarito. A versão final acrescenta ferramentas auxiliares do domínio, como a identificação de nomes e a resolução de períodos, que o modelo consulta em sucessivas chamadas, e uma verificação dos parâmetros que lhe devolve os erros. Foram construídos três conjuntos de consultas, de referência (310, com gabarito auditado), de validação (1.929) e de teste (945, nunca usado no desenvolvimento). Foi desenvolvido um *pipeline* reprodutível para avaliar as versões em quatro modelos locais, executados pelo Ollama, e compará-las com a solução do Maj Diniz e com a saída estruturada (teste A/B, também com três modelos maiores em nuvem). Na primeira versão, com os três melhores modelos locais, o *tool calling* não diferiu significativamente em acurácia da solução do Maj Diniz (58,8% no conjunto de referência) e teve tempo de resposta uma ordem de grandeza menor, pela troca por modelos menores. No conjunto de teste, a versão final com o Gemma 4 E4B obteve acurácia de 94,4%, contra 87,8% do controle, a saída estruturada com as mesmas descrições e a mesma verificação, sem as ferramentas auxiliares e outras peças do *tool calling*. Essa vantagem repetiu-se com o Gemma 4 E2B e inverteu-se com o Qwen 3 4B. No teste A/B, em pares com o mesmo modelo, conjunto e versão, a abordagem mais acurada dependeu da versão e do modelo, e o *tool calling*, sem ganho de tempo, obteve o maior F1 da recusa em todos os pares fora do desenvolvimento. Numa análise complementar no acervo real, as buscas registradas da versão final com o Gemma 4 E4B devolveram as mesmas cartas que o gabarito em 96,7% das consultas de teste com cartas no gabarito. Ressalva-se que as consultas são sintéticas e que as ferramentas e a verificação codificam o mesmo manual que define o gabarito.

Palavras-chave: Modelos amplos de linguagem. *Tool calling*. Catálogo de dados espaciais. Disseminação da informação geoespacial. Cartografia sistemática. Ollama. Avaliação de modelos.

ABSTRACT

The cartographic collection of the Brazilian Army Geographic Service Directorate (DSG), in the Army Geographic Database (BDGEx), is organized by technical nomenclatures, such as the Nomenclature Index and the Index Map, that tend to restrict its use by non-specialists. To disseminate geospatial information, natural-language search is desirable, with a language model translating the free-form query into the criteria of the catalog. The `prototipo_busca_llm` project, developed by Maj Diniz at the 1st Geoinformation Center (1º CGEO), performed this translation with structured output, in which the model fills a fixed format, using a single local model (Phi-4 14B) and a normalization dictionary, without an evaluation protocol. The objective of this work was to adapt that project to tool calling, in which the model chooses whether to call tools and with which arguments. The first of the three versions developed offers only the search tool, in a single model call. The second adds a refusal tool and writes into the tool descriptions conventions of the manual that defines the ground truth. The final version adds auxiliary domain tools, such as name identification and period resolution, which the model consults over successive calls, and a parameter check that returns the errors to it. Three query sets were built, the reference set (310, with an audited ground truth), the validation set (1,929), and the test set (945, never used in development). A reproducible pipeline was developed to evaluate the versions on four local Ollama models and to compare them with Maj Diniz's solution and with structured output (A/B test, also with three larger cloud models). In the first version, with the three best local models, tool calling did not differ significantly in accuracy from Maj Diniz's solution (58.8% on the reference set) and had a response time one order of magnitude shorter, owing to the change to smaller models. On the test set, the final version with Gemma 4 E4B reached an accuracy of 94.4%, against 87.8% for the control, structured output with the same descriptions and the same parameter check, without the auxiliary tools and other parts of tool calling. This advantage recurred with Gemma 4 E2B and reversed with Qwen 3 4B. In the A/B test, in pairs with the same model, query set and version, the more accurate approach depended on the version and the model, and tool calling, with no gain in time, had the higher refusal F1 in every pair outside development. In a complementary analysis on the real collection, the recorded searches of the final version with Gemma 4 E4B returned the same maps as the ground truth in 96.7% of the test queries with maps in the ground truth. As caveats, the queries are synthetic, and the tools and the parameter check encode the same manual that defines the ground truth.

Keywords: Large language models. Tool calling. Spatial data catalog. Geospatial information dissemination. Systematic topographic mapping. Ollama. Model evaluation.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura do projeto `prototipo_busca_llm`, do 1º CGEO 23
Figura 2 – Fluxo do *tool calling* 25
Figura 3 – Componentes da solução e tecnologia de cada um 34
Figura 4 – Sequência metodológica em notação BPMN 37
Figura 5 – Laço de chamadas da versão final da tradução 53

LISTA DE QUADROS

Quadro 1 – Requisitos da solução e a sua origem 32
Quadro 2 – Configurações recomendadas por cenário de implantação 74

LISTA DE TABELAS

Tabela 1 – Consulta técnica e consulta em linguagem natural 15
Tabela 2 – Modelos de linguagem empregados, com porte, licença, lançamento e suporte a *tool calling* 22
Tabela 3 – Ambientes de execução e uso de cada um 36
Tabela 4 – Uso de ferramentas de inteligência artificial por fase da sequência metodológica 47
Tabela 5 – Composição do acervo do BDGEx coletado pelo CSW 49
Tabela 6 – Vícios do acervo e funcionalidade ou limitação relacionada a cada um 50
Tabela 7 – Funcionalidades da busca, erro ou ambiguidade que resolvem, conceito, requisito e versão 59
Tabela 8 – Modelos instalados, memória ocupada e emissão das chamadas de ferramenta 60
Tabela 9 – Conjuntos de consultas construídos para a validação e a avaliação . . 61
Tabela 10 – Primeira versão nos quatro modelos locais, saída estruturada e solução do Maj Diniz no conjunto de referência 62
Tabela 11 – Passos da versão final, controle e outros modelos no conjunto de teste 65
Tabela 12 – Teste A/B entre o *tool calling* e a saída estruturada 67
Tabela 13 – Acerto por resultado e por parâmetros no acervo real do BDGEx . . 69
Tabela 14 – Atendimento dos requisitos 71

LISTA DE ABREVIATURAS E SIGLAS

1º CGEO	1º Centro de Geoinformação
API	*Application Programming Interface* (Interface de Programação de Aplicações)
BDGEx	Banco de Dados Geográficos do Exército
BFCL	*Berkeley Function Calling Leaderboard*
BPMN	*Business Process Model and Notation* (Modelo e Notação de Processos de Negócio)
CGEO	Centro de Geoinformação
CIM	Carta Internacional do Mundo ao Milionésimo
CPU	*Central Processing Unit* (Unidade Central de Processamento)
CSW	*Catalogue Service for the Web*
DSG	Diretoria de Serviço Geográfico
ET-PCDG	Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais
GPU	*Graphics Processing Unit* (Unidade de Processamento Gráfico)
IA	Inteligência artificial
IBGE	Instituto Brasileiro de Geografia e Estatística
IC	Intervalo de confiança
INDE	Infraestrutura Nacional de Dados Espaciais
INOM	Índice de Nomenclatura
JSON	*JavaScript Object Notation*
LLM	*Large Language Model* (Modelo amplo de linguagem)
MDS	Modelo digital de superfície
MDT	Modelo digital de terreno
MGB	Perfil de Metadados Geoespaciais do Brasil

MI	Mapa Índice
OGC	*Open Geospatial Consortium*
PFC	Projeto Final de Curso
p.p.	Pontos percentuais
QAT	*Quantization-Aware Training* (quantização consciente de treinamento)
REST	*Representational State Transfer*
RF	Requisito funcional
RNF	Requisito não funcional
SCN	Sistema Cartográfico Nacional
SIG	Sistema de Informação Geográfica
SQL	*Structured Query Language*
UF	Unidade da Federação
VRAM	*Video Random Access Memory* (memória da GPU)

SUMÁRIO

1 INTRODUÇÃO 14
1.1 Contextualização 14
1.2 Objetivo 17
1.2.1 Objetivo Geral 17
1.2.2 Objetivos Específicos 17
1.2.3 Questões do Trabalho 17
1.3 Estrutura do Texto 18

2 FUNDAMENTAÇÃO TEÓRICA 19
2.1 Modelos Amplos de Linguagem 19
2.1.1 Conceitos 19
2.1.2 Execução Local e Quantização 20
2.1.3 Modelos Empregados 20
2.2 Análise do Projeto prototipo_busca_llm do 1º CGEO 22
2.2.1 Arquitetura 23
2.2.2 Contribuições e Limitações 24
2.3 *Tool Calling* 24
2.3.1 Definição e Fluxo 24
2.3.2 *Tool Calling*, Saída Estruturada e Geração de SQL 26
2.3.3 Ferramentas Auxiliares, Retorno e Decisão de Chamar 27
2.3.4 Avaliação 27
2.3.5 Uso de Ferramentas por Modelos de Linguagem em Geoinformação 29

3 DESENVOLVIMENTO 31
3.1 Requisitos 31
3.2 Ambiente Computacional 33
3.2.1 Componentes da Solução e Tecnologias 33
3.2.2 *Hardware* e Requisitos dos Modelos 35
3.3 Sequência Metodológica 36
3.3.1 Visão Geral das Três Versões 37
3.3.2 Extração dos Dados 38
3.3.3 Modelagem das Funcionalidades 38
3.3.3.1 Tipos de Consulta e Exceções 39
3.3.3.2 Manual de Anotação 39
3.3.3.3 Ciclo de Desenvolvimento 40
3.3.4 Instalação dos Modelos 40

3.3.5 Validação dos Modelos . . . 41
3.3.5.1 Conjunto de Referência . . . 41
3.3.5.2 Conjuntos de Validação e de Teste . . . 41
3.3.5.3 Papéis dos Conjuntos e Congelamento . . . 42
3.3.6 Avaliação de Performance . . . 43
3.3.6.1 Medidas . . . 43
3.3.6.2 Regra de Decisão . . . 43
3.3.6.3 Comparação com a Solução do Maj Diniz . . . 44
3.3.6.4 Passos da Versão Final e Controle . . . 44
3.3.6.5 Teste A/B . . . 45
3.3.6.6 Avaliação pela Execução no Acervo Real . . . 46
3.3.7 Uso de Ferramentas de Inteligência Artificial . . . 46

4 RESULTADOS . . . 48
4.1 Extração dos Dados . . . 48
4.2 Modelagem das Funcionalidades . . . 51
4.2.1 Primeira Versão e a Ferramenta de Busca . . . 51
4.2.2 Segunda Versão, Convenções nas Descrições e Recusa . . . 52
4.2.3 Versão Final . . . 53
4.2.3.1 Descrições Revisadas . . . 54
4.2.3.2 Identificação de Nomes . . . 54
4.2.3.3 Normalização de Códigos . . . 55
4.2.3.4 Normalização de Escalas . . . 55
4.2.3.5 Data de Execução e Resolução de Períodos . . . 55
4.2.3.6 Pedido de Esclarecimento . . . 56
4.2.3.7 Verificação dos Parâmetros com Retorno ao Modelo . . . 57
4.2.3.8 Recusa Contestada . . . 57
4.2.3.9 Desenvolvimento e Congelamento . . . 58
4.2.4 Predicados Espaciais . . . 58
4.2.5 Síntese das Funcionalidades . . . 58
4.3 Instalação . . . 59
4.4 Validação dos Modelos . . . 61
4.5 Avaliação de Performance . . . 63
4.5.1 Comparação com a Solução do Maj Diniz . . . 63
4.5.2 As Versões no Conjunto de Teste . . . 64
4.5.3 Teste A/B . . . 67
4.5.4 Avaliação pela Execução no Acervo Real . . . 68
4.5.5 Tempo de Processamento . . . 70
4.6 Discussão dos Resultados . . . 70
4.6.1 Atendimento dos Requisitos . . . 70

4.6.2 Ganhos Observados 71
4.6.3 Respostas às Questões do Trabalho 72
4.6.4 Recomendação por Cenário 73
4.6.5 Validade dos Resultados 74
4.7 Entregáveis 75

5 CONCLUSÃO 76
5.1 Retomada do Objetivo e Entrega 76
5.2 Contribuições 76
5.3 Limitações 77
5.4 Trabalhos Futuros 78

REFERÊNCIAS 80

1 INTRODUÇÃO

1.1 Contextualização

A informação geoespacial tem papel estratégico na defesa, no planejamento territorial, na gestão ambiental e na tomada de decisão do Estado brasileiro. O Decreto n.º 6.666/2008 instituiu a Infraestrutura Nacional de Dados Espaciais (INDE), que organiza o acesso integrado aos dados geoespaciais produzidos pelos órgãos federais e inclui a disseminação e o uso desses dados entre os seus objetivos (BRASIL, 2008). No Exército Brasileiro, a Diretoria de Serviço Geográfico (DSG) orienta as atividades referentes a imagens e informações geográficas, com vistas à produção cartográfica, ao estabelecimento de normas e ao suprimento do material a seu cargo (DIRETORIA DE SERVIÇO GEOGRÁFICO DO EXÉRCITO BRASILEIRO, 2025b). A DSG mantém o Banco de Dados Geográficos do Exército (BDGEx), criado para disponibilizar na Internet os dados geoespaciais da instituição, conforme o decreto da INDE (XAVIER; MEYER; LUNARDI, 2014), e por meio dele o acervo cartográfico oficial fica acessível ao público militar e à sociedade civil (DIRETORIA DE SERVIÇO GEOGRÁFICO DO EXÉRCITO BRASILEIRO, 2025a). Cinco Centros de Geoinformação (CGEO), sediados em Porto Alegre, Brasília, Olinda, Manaus e Rio de Janeiro, produzem esse acervo segundo a Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG) (BRASIL. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016) e o Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) (BRASIL. Comissão Nacional de Cartografia (CONCAR), 2009), e cada CGEO atende a uma região, a sua área de suprimento. Os produtos são catalogados em serviços compatíveis com os padrões do *Open Geospatial Consortium* (OGC), em particular o *Catalogue Service for the Web* (CSW) (OPEN GEOSPATIAL CONSORTIUM, 2007), e o catálogo organiza os metadados num modelo de dados rígido.

A busca nesse catálogo exige que o usuário conheça as convenções técnicas que organizam o acervo. Ele precisa identificar a folha, unidade da articulação sistemática das cartas, pelo Índice de Nomenclatura (INOM), código que subdivide a folha da Carta Internacional do Mundo ao Milionésimo até a escala pedida, ou pelo Mapa Índice (MI), que numera as folhas de cada escala (IBGE, 1999). Precisa também saber quais escalas o catálogo oferece, conhecer os tipos de produto do Sistema Cartográfico Nacional (SCN) (BRASIL, 1967), como “SCN Carta Topográfica Matricial”, e reconhecer o CGEO responsável por cada folha. Esses conceitos são detalhados na Seção 4.2, junto das ferramentas que os codificam. A interface de busca do catálogo compara o texto digitado com os campos do banco, sem interpretar a linguagem natural, de modo que só encontra o produto quem escreve os termos técnicos do acervo. A Tabela 1 compara esse modo de consulta

com o modo desejado.

Tabela 1 – Consulta técnica e consulta em linguagem natural

Modo atual (busca por texto simples)	**Modo desejado (linguagem natural)**
O usuário precisa digitar os termos técnicos que coincidem com os campos do banco, como "SCN Carta Topográfica Matricial 1:25.000 Rio de Janeiro 2023"	*"mapas do Rio de Janeiro em 25k publicados em 2023"*

Fonte: Adaptado de 1º Centro de Geoinformação (2025).

Essa barreira técnica restringe o uso do acervo por analistas, gestores e tomadores de decisão que precisam de informação geoespacial, mas não têm formação em cartografia, e problema análogo é relatado na descoberta de dados em geoportais e em catálogos de dados espaciais em geral (HU et al., 2015; LI et al., 2025). Por essa razão, no âmbito da Engenharia Cartográfica, o trabalho situa-se na disseminação da informação geoespacial, etapa que torna o acervo produzido acessível a quem dele necessita.

A solução desejável para esse problema é uma busca em linguagem natural apoiada em modelos amplos de linguagem, designados no restante do texto como modelos de linguagem, que interpretam intenções implícitas, normalizam variações de escrita e traduzem descrições informais em estruturas formais (BROWN et al., 2020; WEI et al., 2022). Nessa busca, o modelo traduz a consulta livre do usuário nos critérios do catálogo, e o sistema executa a busca com esses critérios. Duas formulações permitem obter dos modelos critérios de busca. No *tool calling*, o modelo recebe a descrição de funções externas, denominadas ferramentas, e escolhe se chama alguma delas e com quais argumentos. O sistema que hospeda o modelo executa a ferramenta chamada e pode devolver o resultado ao modelo antes da resposta final (SCHICK et al., 2023; QIN et al., 2024). Na saída estruturada, o modelo devolve sempre um objeto num formato predefinido pela aplicação, com os campos que preencheu, e o objeto pode ser conferido contra esse formato (TAM et al., 2024; GENG et al., 2025). As duas formulações são detalhadas na Seção 2.3.

Para tratar o problema, o 1º Centro de Geoinformação (1º CGEO), na sua Divisão de Levantamento, desenvolveu o projeto `prototipo_busca_llm` (1º CENTRO DE GEOINFORMAÇÃO, 2025), de autoria do Maj Diniz. O projeto implementa uma busca em linguagem natural sobre o catálogo com o modelo Phi-4 14B (ABDIN et al., 2024) e saída estruturada, com um dicionário de normalização que uniformiza termos da consulta antes de ela chegar ao modelo. O modelo é executado localmente com o Ollama (OLLAMA, 2023), plataforma de execução de modelos abertos no próprio *hardware*, e a execução local atende às restrições de sigilo da informação do Exército e dispensa o custo recorrente

dos serviços em nuvem. O projeto demonstrou a viabilidade da abordagem no domínio cartográfico nacional e é o ponto de partida deste trabalho. O seu método, reproduzido neste trabalho, é designado no texto como solução do Maj Diniz e constitui a referência de comparação dos resultados. A arquitetura, as contribuições e as limitações do projeto são analisadas na Seção 2.2.

O *tool calling* tem potencial para aprimorar essa solução, e a sua adoção consta das demandas que o 1º CGEO apresentou para este trabalho. Com ele, o modelo pode recusar, sem chamar a ferramenta de busca, o pedido que não pertence ao acervo, ao passo que, na saída estruturada, toda consulta resulta num objeto de busca, e a recusa exige um campo criado para esse fim no formato. O modelo pode também chamar ferramentas auxiliares do domínio antes de buscar, como a lista de municípios, o índice de folhas e as regras de datas, e receber o retorno da verificação dos parâmetros como resultado da própria chamada de ferramenta, enquanto, na saída estruturada, esse retorno só chega ao modelo como nova tentativa, depois de emitido o objeto inteiro. Essas vantagens são hipóteses, discutidas nas Seções 2.3.1 e 2.3.3, que precisam ser testadas contra a saída estruturada nos mesmos modelos e nas mesmas consultas.

O objeto do trabalho é a tradução da consulta em linguagem natural nos parâmetros de busca do catálogo. A tradução é avaliada contra um gabarito, a resposta correta de cada consulta expressa em parâmetros, e a busca que esses parâmetros produzem é verificada também no acervo real do BDGEx. A busca restringe os resultados pela interseção espacial entre a folha e o estado, o município ou a área de suprimento citados na consulta, filtro que, no acervo real, foi executado com as malhas de estados e municípios do Instituto Brasileiro de Geografia e Estatística (IBGE) e com uma aproximação das áreas de suprimento, que não têm polígono oficial (Seção 4.2.4). Os predicados espaciais relacionais, como “perto de”, “num raio de” e “ao norte de”, são excluídos do escopo e indicados como trabalho futuro (Seção 5.4). A sua implementação exigiria geocodificar o lugar citado, isto é, obter a sua posição a partir do nome, implementar operadores de distância e de direção e acrescentar novos parâmetros aos doze parâmetros de busca do `prototipo_busca_llm`, que foram mantidos para que a solução desenvolvida e a solução do Maj Diniz fossem avaliadas contra o mesmo gabarito. Também estão fora do escopo:

- a interface gráfica para o usuário final;
- a implantação em produção na DSG;
- a avaliação com usuários e a pesquisa de opinião, previstas nas demandas do 1º CGEO e indicadas como trabalho futuro;
- o treinamento e o ajuste fino de modelos de linguagem;
- a busca por coordenadas sem referência nominal a um lugar;

- idiomas além do português.

1.2 Objetivo

1.2.1 Objetivo Geral

O objetivo geral do trabalho é adequar o projeto `prototipo_busca_llm`, do 1º CGEO, para empregar os recursos de *tool calling* na tradução de consultas em linguagem natural para critérios de busca no acervo cartográfico da DSG, com a finalidade de facilitar a disseminação desse acervo a usuários não especialistas.

1.2.2 Objetivos Específicos

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

1. reescrever o *backend* do projeto para empregar o *tool calling* nativo, no qual o servidor de modelos devolve as chamadas de ferramenta num campo próprio da resposta (Seção 2.3.1);
2. modelar as funcionalidades da busca como ferramentas do domínio, que codificam as convenções da cartografia sistemática e do catálogo;
3. avaliar modelos de linguagem abertos executados localmente quanto à acurácia, definida como a proporção de consultas traduzidas corretamente, e quanto ao tempo de processamento, e comparar a solução desenvolvida com a solução do Maj Diniz e, num teste A/B, com a saída estruturada executada nos mesmos modelos e nas mesmas consultas;
4. disponibilizar o código, os conjuntos de consultas construídos para a avaliação e os registros das avaliações.

1.2.3 Questões do Trabalho

A avaliação foi planejada para responder às seguintes questões:

1. O *tool calling* melhora a tradução em relação à solução do Maj Diniz?
2. No teste A/B, o *tool calling* obtém acurácia maior que a da saída estruturada?
3. No teste A/B, o *tool calling* reduz o tempo de processamento em relação à saída estruturada?
4. As ferramentas auxiliares do domínio aumentam a acurácia dos modelos locais?

As respostas estão na Seção 4.6.3, na mesma ordem. Como análise complementar, sem novas rodadas de modelos, verificou-se também se as buscas produzidas devolvem no acervo real do BDGEx as mesmas cartas que as buscas do gabarito (Seção 4.5.4). Essa análise, que não integra as questões planejadas do trabalho, segue o princípio da avaliação pela execução (Seção 2.3.4), no qual se compara o resultado da busca, e não apenas os parâmetros.

1.3 Estrutura do Texto

O texto está organizado em cinco capítulos. O Capítulo 2 apresenta os modelos de linguagem empregados, a análise do projeto `prototipo_busca_llm` e o *tool calling*, com os termos usados na metodologia e na avaliação. O Capítulo 3 define os requisitos, o ambiente computacional e a sequência metodológica, com o uso de ferramentas de inteligência artificial em cada fase. O Capítulo 4 descreve os dados extraídos do acervo, as funcionalidades da busca e o raciocínio que motivou cada ferramenta, a instalação, a validação e a avaliação dos modelos, a discussão dos resultados e os entregáveis. O Capítulo 5 retoma o objetivo e apresenta as contribuições, as limitações e os trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os modelos de linguagem empregados, com os parâmetros que os distinguem e os requisitos para executá-los, analisa o projeto `prototipo_busca_llm`, do 1º CGEO, e define o *tool calling* e os termos usados na metodologia e na avaliação dos resultados.

2.1 Modelos Amplos de Linguagem

2.1.1 Conceitos

Os modelos amplos de linguagem (LLM, do inglês *Large Language Models*), também denominados modelos de linguagem de grande porte, como no título deste trabalho, são redes neurais baseadas na arquitetura *Transformer* (VASWANI et al., 2017), treinadas em grandes volumes de texto. Eles geram a resposta um *token* (fragmento de palavra) por vez, e cada *token* é condicionado às instruções recebidas, à consulta e ao que o próprio modelo já gerou. Com o aumento da escala do modelo e dos dados de treinamento, surgem capacidades como a de interpretar intenções implícitas (BROWN et al., 2020; WEI et al., 2022).

O porte de um modelo é expresso pelo número de parâmetros, os pesos numéricos aprendidos no treinamento, contado em bilhões (B), como no Qwen 3 4B. Nas arquiteturas densas, todos os parâmetros participam do cálculo de cada *token*. Nas arquiteturas de mistura de especialistas, apenas uma parte deles, os parâmetros ativos, é usada em cada *token*, o que reduz o custo da geração, mas não a memória necessária para armazenar o modelo. Em alguns modelos densos, distinguem-se ainda os parâmetros efetivos dos parâmetros totais armazenados. Os efetivos excluem as tabelas de *embeddings* (representações vetoriais dos *tokens*) mantidas para cada camada. A Seção 2.1.3 apresenta exemplos dos três casos.

A capacidade de seguir instruções em linguagem natural é obtida pelo ajuste fino com instruções e com retorno humano (OUYANG et al., 2022). O *tool calling*, definido na Seção 2.3.1, depende dessa capacidade, porque o modelo precisa entender a consulta, interpretar o que cada ferramenta disponível faz e preencher os parâmetros dela com valores coerentes com o pedido.

Dois controles da geração são relevantes para a metodologia. O primeiro é o modo de raciocínio, presente em parte dos modelos recentes, no qual o modelo gera um texto intermediário antes da resposta. Esse modo pode ajudar em escolhas que combinam vários critérios, mas consome mais *tokens* e mais tempo (QWEN TEAM, 2025b). O segundo é

a temperatura, que controla a aleatoriedade na escolha de cada *token*. Com temperatura zero, o modelo escolhe sempre o *token* mais provável, e a mesma entrada tende a produzir a mesma saída.

2.1.2 Execução Local e Quantização

A execução dos modelos em servidores da própria organização traz vantagens a instituições como a DSG. As consultas não trafegam por redes externas, não há custo por *token* processado, a organização mantém a versão exata do modelo, que pode ser auditada, e o sistema funciona sem Internet. Há também desvantagens. A primeira é que os modelos de fronteira, de pesos fechados e por isso indisponíveis para execução local, ocupam a maior parte das primeiras posições em avaliações de uso de ferramentas, como o *Berkeley Function Calling Leaderboard* (BFCL) (PATIL et al., 2025). A segunda é que o custo passa a ser o do *hardware*, isto é, o de unidades de processamento gráfico (GPU, do inglês *Graphics Processing Unit*), da memória delas (VRAM, do inglês *Video Random Access Memory*) e de energia. A terceira é que cada modelo novo exige repetir a avaliação.

O Ollama (OLLAMA, 2023) é uma plataforma de código aberto que instala e executa modelos de linguagem localmente e os expõe por uma interface de programação de aplicações (API, do inglês *Application Programming Interface*) compatível com a da OpenAI. Os modelos são armazenados no formato de arquivo GGUF (GGML CONTRIBUTORS, 2023), que reúne os pesos e os metadados do modelo e admite várias variantes de quantização. A quantização reduz a precisão dos pesos de 16 para cerca de 4 bits e, com isso, a memória que eles ocupam, com pequena perda de qualidade (FRANTAR et al., 2023). Na quantização consciente de treinamento (QAT, do inglês *Quantization-Aware Training*), o modelo é ajustado durante o treinamento para operar nessa precisão (GOOGLE DEEPMIND, 2026).

A memória ocupada por um modelo em execução soma a dos pesos e a do contexto, que cresce com o texto processado. Quando o modelo não cabe na VRAM, parte dele é executada na unidade central de processamento (CPU, do inglês *Central Processing Unit*), num arranjo chamado *offload*, que aumenta o tempo de resposta. O tamanho do arquivo é, por isso, apenas um indicador inicial do *hardware* necessário.

O catálogo do Ollama indica, para cada modelo, se ele oferece *tool calling* nativo, recurso definido na Seção 2.3.1.

2.1.3 Modelos Empregados

A Tabela 2 reúne os modelos empregados, agrupados pelo papel que desempenham. Entre os quatro modelos locais avaliados, o Qwen 3 4B Instruct 2507, da equipe Qwen, da Alibaba Cloud (QWEN TEAM, 2025a), é a variante sem modo de raciocínio de um

modelo denso de 4 bilhões de parâmetros da família Qwen3 (QWEN TEAM, 2025b). O Gemma 4 E4B e o Gemma 4 E2B, da Google DeepMind (GEMMA TEAM, 2026), têm, respectivamente, 4,5 e 2,3 bilhões de parâmetros efetivos, ou 8 e 5,1 bilhões quando se contam as tabelas de *embeddings* por camada, e foram empregados nas versões com QAT (GOOGLE DEEPMIND, 2026). O Mistral Nemo 12B, desenvolvido pela Mistral AI com a NVIDIA (MISTRAL AI, 2024), é um modelo denso de 12 bilhões de parâmetros, o maior dos quatro modelos locais avaliados.

O Phi-4 14B, da Microsoft Research (ABDIN et al., 2024), sob licença MIT (MICROSOFT RESEARCH, 2024), é o modelo do projeto `prototipo_busca_llm` e, por isso, da solução do Maj Diniz (Seção 3.3.6.3). O catálogo do Ollama não indica *tool calling* nativo para ele, condição compatível com o emprego da saída estruturada no projeto (Seção 2.2).

Três modelos maiores foram executados por meio do Groq, serviço de inferência em nuvem que executa modelos de pesos abertos. O GPT-OSS 20B e o GPT-OSS 120B, da OpenAI (OPENAI, 2025), são modelos de mistura de especialistas, com 21 bilhões de parâmetros totais e 3,6 ativos no primeiro e 117 totais e 5,1 ativos no segundo. Em ambos, o modo de raciocínio não pode ser desligado (GROQ, 2026). O Qwen 3.8 27B, da equipe Qwen (QWEN TEAM, 2026), representa a geração mais recente dessa família. A menor versão dele no catálogo do Ollama tem 27 bilhões de parâmetros e ocupa 18 GB, mais do que a VRAM das GPUs disponíveis para a execução local (Seção 3.2.2), e, por isso, essa geração foi avaliada apenas em nuvem.

Todos os modelos têm pesos abertos. O Phi-4 14B é distribuído sob a licença MIT e os demais sob a Apache 2.0, ambas permissivas quanto ao uso, à modificação e à redistribuição. Os lançamentos vão de julho de 2024 (Mistral Nemo 12B) a agosto de 2026 (Qwen 3.8 27B). Entre o lançamento da família Qwen3, em abril de 2025, e o do Gemma 4, em abril de 2026, passou-se cerca de um ano, e a variante Qwen 3 4B Instruct 2507 é de agosto de 2025. A data é registrada porque o ecossistema de modelos evolui rapidamente. A relação entre o lançamento, o porte e o desempenho dos modelos avaliados é discutida na Seção 4.4.

A janela de contexto oficial, que limita o texto processado de uma vez, é de 128 mil *tokens* no Gemma 4 e no Mistral Nemo 12B, de cerca de 262 mil no Qwen 3 4B Instruct 2507 e de 16 mil no Phi-4 14B. O ambiente escolhido para cada modelo é justificado na Seção 3.2.2.

Tabela 2 – Modelos de linguagem empregados, com porte, licença, lançamento e suporte a *tool calling*

Modelo	**Organização**	**Parâmetros**	**Arquivo no Ollama**	**Licença**	**Lançamento**	***Tool calling* nativo no Ollama**
Modelos locais avaliados						
Qwen 3 4B Instruct 2507 (QWEN TEAM, 2025a)	Equipe Qwen (Alibaba Cloud)	4,0 bilhões	2,5 GB	Apache 2.0	ago. 2025	sim
Gemma 4 E4B (GEMMA TEAM, 2026)	Google DeepMind	4,5 bilhões efetivos (8 com *embeddings*)	6,1 GB (QAT)	Apache 2.0	abr. 2026 (QAT em jun. 2026)	sim
Gemma 4 E2B (GEMMA TEAM, 2026)	Google DeepMind	2,3 bilhões efetivos (5,1 com *embeddings*)	4,3 GB (QAT)	Apache 2.0	abr. 2026 (QAT em jun. 2026)	sim
Mistral Nemo 12B (MISTRAL AI, 2024)	Mistral AI e NVIDIA	12 bilhões	7,1 GB	Apache 2.0	jul. 2024	sim
Modelo da solução do Maj Diniz						
Phi-4 14B (ABDIN et al., 2024)	Microsoft Research	14 bilhões	9,1 GB	MIT	dez. 2024	não
Modelos maiores em nuvem (Groq)						
GPT-OSS 20B (OPENAI, 2025)	OpenAI	21 bilhões (3,6 ativos)	não instalado	Apache 2.0	ago. 2025	sim
GPT-OSS 120B (OPENAI, 2025)	OpenAI	117 bilhões (5,1 ativos)	não instalado	Apache 2.0	ago. 2025	sim
Qwen 3.8 27B (QWEN TEAM, 2026)	Equipe Qwen (Alibaba Cloud)	27 bilhões	não instalado (18 GB no Ollama)	Apache 2.0	ago. 2026	sim

Fonte: Elaborada pelos autores a partir das páginas dos modelos e do catálogo do Ollama (OLLAMA, 2025b; OLLAMA, 2026; OLLAMA, 2024; FARABET; LACOMBE, 2026; MICROSOFT RESEARCH, 2024).

Nota: Tamanho do arquivo em GB decimais, como exibido pelo Ollama, com os pesos do modelo quantizados em cerca de 4 bits. Nos Gemma 4, o arquivo inclui também o projetor de visão e áudio, com cerca de 1 GB, em 16 bits. QAT indica quantização consciente de treinamento.

2.2 Análise do Projeto prototipo_busca_llm do 1º CGEO

O projeto `prototipo_busca_llm` (1º CENTRO DE GEOINFORMAÇÃO, 2025), do 1º CGEO, apresentado na Seção 1.1, é uma prova de conceito de busca no catálogo de

produtos cartográficos com modelo de linguagem executado localmente, e o seu código é público.

2.2.1 Arquitetura

A Figura 1 mostra as quatro camadas da arquitetura.

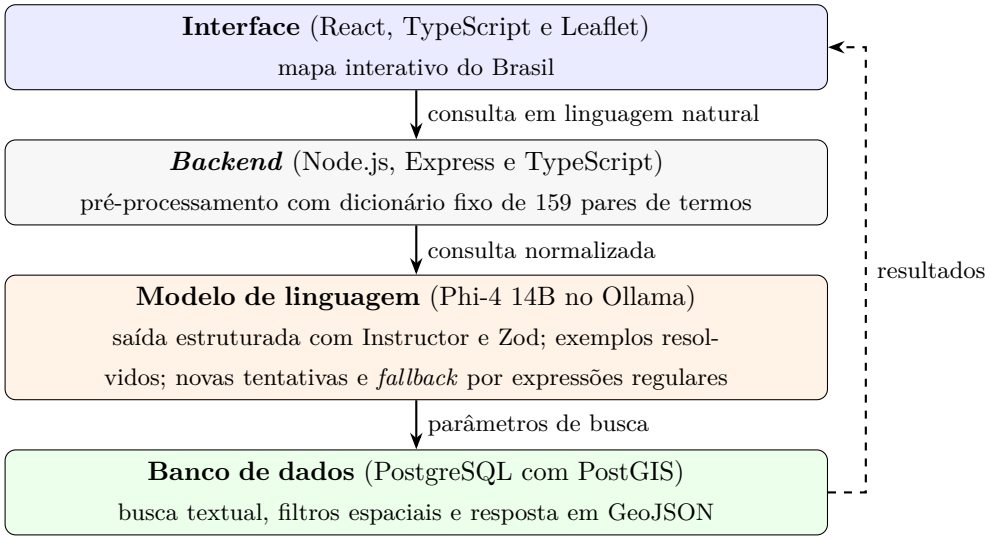


Figura 1 – Arquitetura do projeto `prototipo_busca_llm`, do 1º CGEO

Fonte: Adaptado de 1º Centro de Geoinformação (2025).

A interface, escrita em React com a biblioteca Leaflet, exibe os resultados num mapa interativo do Brasil. O *backend*, em Node.js com Express, recebe a consulta em linguagem natural numa rota de busca. Antes de a consulta chegar ao modelo, um pré-processador aplica um dicionário fixo de 159 pares de substituição, que uniformiza variações de escrita, como a conversão de "25k" em "1:25.000". O núcleo do sistema é o Phi-4 14B, executado pelo Ollama, com saída estruturada obtida pelas bibliotecas Instructor (LIU, 2023) e Zod. O *schema*, especificação dos campos que a resposta do modelo deve conter e do tipo de cada um, inclui um campo de justificativa, em que o modelo explica a extração, e doze parâmetros de busca, a saber, palavra-chave, escala, tipo de produto, estado, município, área de suprimento, projeto, período de publicação, período de criação, campo de ordenação, direção da ordenação e limite de resultados. O *prompt* traz exemplos resolvidos. Se o modelo falhar mesmo depois de três novas tentativas, parte dos campos é extraída do texto por um mecanismo de contingência (*fallback*) baseado em expressões regulares. Os parâmetros obtidos compõem a consulta ao banco PostgreSQL com a extensão PostGIS (POSTGIS DEVELOPMENT TEAM, 2001), que combina busca textual e filtros espaciais e devolve os resultados em GeoJSON. O repositório inclui o esquema do banco, com quatro tabelas, e 22 casos de teste, mas não os dados do acervo.

2.2.2 Contribuições e Limitações

O projeto mostrou que modelos de linguagem executados localmente são tecnicamente viáveis no domínio cartográfico, que o Phi-4 14B extrai parâmetros de busca relevantes de consultas em português e que exemplos resolvidos combinados com saída estruturada produzem resultados utilizáveis. Mostrou também que a integração do modelo com o PostgreSQL e o PostGIS funciona em ambiente controlado.

Na apresentação das demandas do trabalho, o 1º CGEO apontou seis limitações do projeto, enumeradas a seguir na ordem em que os requisitos do Capítulo 3 as referenciam:

1. o *backend* foi escrito em Node.js, embora as bibliotecas de integração com modelos de linguagem tenham versões mais completas em Python;
2. a tradução da consulta usa apenas saída estruturada, de modo que o modelo devolve sempre a estrutura predefinida, não pode recusar um pedido sem um campo próprio para isso e não pode consultar funções durante a resposta;
3. um único modelo é empregado, sem comparação com alternativas, num ecossistema de modelos em rápida evolução;
4. o dicionário fixo no código deixa sem normalização, e sem aviso, as variações não previstas, como "dois cgeo" ou "25 mil", exige manutenção manual e pode deturpar uma consulta legítima quando um termo do dicionário aparece em contexto ambíguo;
5. o mecanismo de contingência por expressões regulares encobre falhas do modelo que não foram documentadas de forma sistemática, o que dificulta a análise das suas causas;
6. faltam métricas de avaliação, pois os 22 casos de teste são pontuados por pesos atribuídos *ad hoc*, sem penalizar campos preenchidos indevidamente e sem protocolo para comparar modelos.

2.3 *Tool Calling*

Esta seção define o *tool calling*, compara-o com as formulações alternativas e reúne a literatura e os conceitos de avaliação empregados na metodologia.

2.3.1 Definição e Fluxo

O *tool calling* é a capacidade de um modelo de linguagem de reconhecer que a resposta a uma consulta exige uma função externa e de gerar a chamada dessa função, com os argumentos em JSON (do inglês *JavaScript Object Notation*) (SCHICK et al.,

2023; QIN et al., 2024). O modelo escolhe a função e os argumentos, e o sistema que o hospeda executa a função escolhida, em geral determinística, como uma consulta a banco de dados, uma chamada a outra API ou a execução de um código.

Os termos a seguir são usados no restante do texto. Uma **ferramenta** é uma função externa oferecida ao modelo. A **descrição da ferramenta**, escrita na notação JSON Schema, informa o nome, a finalidade e os parâmetros da ferramenta, cada um com o tipo, uma descrição e, quando cabe, o **enumerado**, isto é, a lista fechada de valores válidos. Por especificar os campos e os tipos dos argumentos, a descrição é também chamada de *schema* da ferramenta. A **chamada de ferramenta** é o pedido, emitido pelo modelo, de execução de uma ferramenta com certos argumentos. A **chamada ao modelo** é uma ida e volta ao modelo, na qual o sistema envia as mensagens e recebe uma resposta. O **resultado da ferramenta** é o que ela devolve e que retorna ao modelo como nova mensagem. O **laço de chamadas** é a sequência de chamadas ao modelo até a resposta final. O ***prompt* de sistema** reúne as instruções fixas que a aplicação envia ao modelo. Quando o modelo foi ajustado para emitir chamadas de ferramenta, o servidor as devolve num campo próprio da resposta, separado do texto, o **canal de ferramentas**, e esse arranjo caracteriza o ***tool calling* nativo**. A Figura 2 mostra o fluxo.

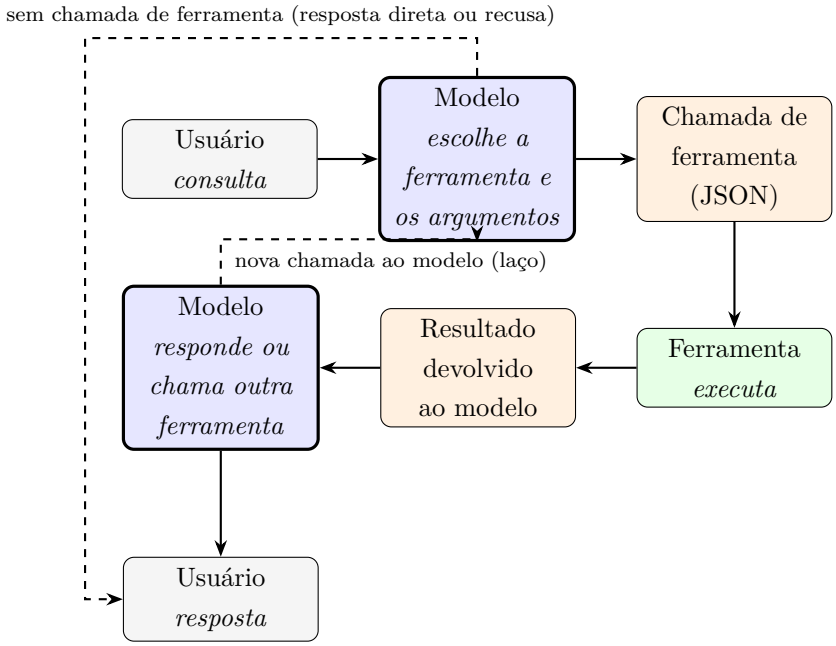


Figura 2 – Fluxo do *tool calling*

Fonte: Elaborado pelos autores, com base em Schick et al. (2023) e Qin et al. (2024).

Do fluxo da Figura 2 decorrem as vantagens que motivam a adoção do *tool calling*. A ausência de chamada de ferramenta é uma resposta válida, de modo que o modelo pode recusar um pedido fora do escopo sem que o formato da resposta preveja um campo para esse fim. Como os resultados das ferramentas retornam ao modelo como mensagens,

ele pode escolher entre várias ferramentas, consultar funções determinísticas e corrigir os argumentos antes da resposta final. Por fim, as chamadas de ferramenta e os seus argumentos, emitidos em mensagens próprias, ficam disponíveis para auditoria. Essas vantagens dependem da capacidade do modelo de usar as ferramentas oferecidas, discutida na Seção 2.3.3.

2.3.2 *Tool Calling*, Saída Estruturada e Geração de SQL

Na saída estruturada, o modelo devolve a resposta num objeto JSON que segue um *schema* predefinido. A conformidade do objeto ao *schema* é conferida com bibliotecas como o Pydantic (COLVIN, 2019) e o Zod, e, com o Instructor (LIU, 2023), a chamada ao modelo é repetida quando o objeto é inválido. Essa formulação, designada *text-to-JSON* por analogia com o *text-to-SQL* tratado adiante, foi estudada quanto ao efeito das restrições de formato sobre o desempenho. Tam et al. (2024) observaram piora em tarefas de raciocínio quando o formato é imposto com rigidez, resultado competitivo em tarefas de classificação e influência da ordem dos campos sobre o que o modelo produz. Geng et al. (2025) apontam a decodificação restrita ao *schema* como a técnica dominante para impor o formato, mas mostram que a cobertura dos recursos do JSON Schema varia entre as implementações e que a conformidade ao *schema* não é garantida.

Duas limitações estruturais distinguem a saída estruturada do *tool calling*. A primeira é que ela devolve sempre um objeto no formato do *schema*, de modo que toda consulta se converte numa busca e a recusa exige um campo criado para esse fim no próprio *schema*. A segunda é que uma verificação externa só alcança o modelo como nova tentativa, depois de emitido o objeto inteiro. Existe ainda o risco de o modelo preencher campos sem apoio na consulta apenas para completar o *schema*, hipótese que só a avaliação pode confirmar ou afastar.

Na formulação *text-to-SQL*, o modelo gera a consulta inteira, na linguagem de consulta estruturada SQL (do inglês *Structured Query Language*), a partir da pergunta e do esquema do banco de dados. O Spider (YU et al., 2018) consolidou a avaliação dessa tarefa, com cerca de dez mil perguntas sobre 200 bancos. Com os modelos de linguagem, a ambiguidade e a subespecificação das perguntas passaram a figurar entre os problemas centrais da tarefa (LIU et al., 2025). A acurácia ainda é limitada em bancos reais. No BIRD, que reúne bancos grandes e com valores imperfeitos, o GPT-4 acertou o resultado de cerca de 55% das perguntas, contra cerca de 93% obtidos por avaliadores humanos (LI et al., 2023a). Há também um risco de segurança. Pedro et al. (2025) mostraram que, em aplicações construídas com o LangChain (CHASE, 2022) e o LlamaIndex, um *prompt* malicioso pode induzir o modelo a gerar consultas SQL maliciosas, executadas em seguida no banco. O ataque foi confirmado em vários modelos e em aplicações reais, e as defesas baseadas apenas em instruções no *prompt* mostraram-se frágeis.

Uma alternativa à geração de consultas SQL pelo modelo é traduzir a pergunta nos parâmetros tipados de uma ferramenta fixa, ficando a montagem da consulta SQL a cargo do sistema. A escolha adotada, com as vantagens e o custo dessa formulação, está na Seção 3.2.1.

2.3.3 Ferramentas Auxiliares, Retorno e Decisão de Chamar

No ReAct (YAO et al., 2023), o modelo alterna passos de raciocínio e ações, e o resultado de cada ação alimenta o passo seguinte. A literatura indica que a revisão da resposta pelo próprio modelo é eficaz quando o sinal de erro vem de fora dele. O CRITIC (GOU et al., 2024) verifica a saída com ferramentas externas e a revisa com base no resultado, e o *self-debugging* (CHEN et al., 2024) corrige consultas e programas a partir do resultado da execução. Já a autocorreção sem sinal externo (MADAAN et al., 2023) pode não ajudar e até piorar o raciocínio (HUANG et al., 2024). Ferramentas auxiliares são ferramentas que não executam a tarefa final, mas preparam ou conferem, de forma determinística, os argumentos da ferramenta que a executa. Elas e o retorno dos erros ao modelo são, por isso, maneiras de oferecer esse sinal externo.

Avaliações recentes identificam onde os modelos falham no uso de ferramentas. O ToolSandbox (LU et al., 2025) aponta a normalização de argumentos, como datas relativas e nomes de lugares, entre as maiores dificuldades. O When2Call (ROSS; MAHABALESHWARKAR; SUHARA, 2025) mostra que escolher entre chamar uma ferramenta, perguntar ao usuário ou recusar o pedido é uma capacidade própria, fraca em modelos pequenos. O τ-bench (YAO et al., 2025) avalia diálogos em que um usuário simulado, também conduzido por um modelo de linguagem, interage com o modelo avaliado, e o CRITICTOOL (HUANG et al., 2025) classifica os erros de chamada que o modelo precisa perceber e corrigir.

Ajustar um sistema a partir dos erros observados num conjunto rotulado traz o risco de sobreajuste, isto é, de o sistema apresentar bom desempenho apenas nesse conjunto. O risco aplica-se às lições que o próprio modelo acumula a partir de sucessos e falhas, como no Reflexion (SHINN et al., 2023) e no ExpeL (ZHAO et al., 2024), e para a otimização automática de *prompts* (YANG et al., 2024; KHATTAB et al., 2024). A estimativa do desempenho exige, portanto, um conjunto de avaliação que não tenha sido consultado durante o desenvolvimento, pois o reúso do mesmo conjunto ao longo do desenvolvimento compromete a validade das conclusões (DWORK et al., 2015).

2.3.4 Avaliação

Os conjuntos de avaliação de uso de ferramentas, como o API-Bank (LI et al., 2023b), o ToolBench (QIN et al., 2024), com mais de 16 mil APIs reais, e o APIBench,

do Gorilla (PATIL et al., 2024), estão, em geral, em inglês, cobrem domínios amplos e oferecem muitas ferramentas. O BFCL (PATIL et al., 2025) compara, pela árvore sintática, a chamada gerada com a esperada e avalia também a abstenção, quando o correto é não chamar ferramenta alguma.

Em *text-to-SQL*, comparar o texto da consulta gerada com o da consulta de referência é insuficiente, porque mais de uma consulta pode estar correta. Por isso o BIRD e o SQL-Eval (DEFOG, 2023) adotam a avaliação pela execução, também disponível no Spider, em que as duas consultas são executadas no banco e os resultados são comparados. O SQL-Eval aceita a igualdade exata ou por subconjunto dos resultados e adverte que bancos vazios produzem falsos acertos.

A acurácia por consulta é a proporção de consultas completamente corretas,

$$\text{Acurácia} = \frac{N_{\text{corretas}}}{N},$$

em que N_{corretas} é o número de consultas cuja resposta coincide integralmente com a esperada, inclusive quando o correto é não chamar ferramenta alguma, e N é o total de consultas. O critério operacional de acerto é definido na Seção 3.3.6.1.

O acerto parcial, campo a campo, é medido pela precisão (P), pelo *recall* (R) e pelo F1 (MANNING; RAGHAVAN; SCHÜTZE, 2008),

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R},$$

em que TP (verdadeiros positivos) conta os itens produzidos e corretos, FP (falsos positivos) os produzidos e incorretos ou indevidos e FN (falsos negativos) os esperados e não produzidos. As mesmas medidas aplicam-se a uma escolha binária, como recusar ou não um pedido. A acurácia por consulta exige a resposta integralmente correta, ao passo que o F1 por campo credita o acerto parcial.

A concordância entre dois anotadores é medida pelo kappa de Cohen (COHEN, 1960; ARTSTEIN; POESIO, 2008),

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

em que p_o é a concordância observada e p_e a esperada ao acaso, calculada pelas frequências marginais de cada anotador.

Para uma proporção $\hat{p}$ observada em n casos, o intervalo de confiança de Wilson (WILSON, 1927) é

$$\frac{\hat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}},$$

em que z é o quantil da distribuição normal padrão para o nível de confiança, igual a 1,96 para 95%. Ao contrário do intervalo normal simples, o de Wilson permanece entre 0 e 1 mesmo para proporções próximas dos extremos.

Duas configurações avaliadas nas mesmas consultas são comparadas pelo teste exato de McNemar (MCNEMAR, 1947; DIETTERICH, 1998). Apenas os pares discordantes contam, sendo b o número de consultas acertadas só pela primeira configuração e c o número de consultas acertadas só pela segunda. Sob a hipótese de igualdade de desempenho, b segue uma distribuição binomial com $n = b + c$ tentativas e probabilidade $1/2$. O p-valor é a probabilidade de, sob essa hipótese, observar uma diferença igual ou maior do que a observada, e, no teste bicaudal, vale

$$p = \min\left(1,\ 2\sum_{k=0}^{\min(b,c)} \binom{n}{k}\left(\frac{1}{2}\right)^{n}\right).$$

A diferença é considerada significativa quando o p-valor é inferior ao nível de significância α fixado.

O *bootstrap* pareado (EFRON; TIBSHIRANI, 1993) estima a incerteza da diferença de acurácia entre duas configurações. As consultas são reamostradas com reposição muitas vezes, mantendo-se juntos os resultados das duas configurações em cada consulta, e a diferença é recalculada a cada reamostragem. O intervalo de 95% é formado pelos percentis de 2,5 e 97,5 das diferenças obtidas.

Quando m hipóteses são testadas em conjunto, a correção de Holm (HOLM, 1979) controla a probabilidade de ao menos uma rejeição indevida. Os p-valores são ordenados, $p_{(1)} \leq \cdots \leq p_{(m)}$, e a i-ésima hipótese é rejeitada se $p_{(i)} \leq \alpha/(m - i + 1)$. As hipóteses são examinadas nessa ordem até a primeira que não satisfaça a condição, e essa e as seguintes não são rejeitadas.

2.3.5 Uso de Ferramentas por Modelos de Linguagem em Geoinformação

O uso de ferramentas por modelos de linguagem já alcançou a geoinformação. Os modelos passaram a planejar e executar, com alta autonomia, tarefas de Sistemas de Informação Geográfica (SIG). O LLM-Geo (LI; NING, 2023), construído sobre o GPT-4, gera e executa o código que coleta, analisa e visualiza dados espaciais, e o GeoGPT (ZHANG et al., 2024) planeja e encadeia ferramentas de SIG a partir de pedidos em linguagem natural. Em perspectiva mais ampla, Mai et al. (2024) discutem as oportunidades e os desafios dos modelos de fundação para a inteligência artificial (IA) geoespacial, e Manvi et al. (2024) mostram que o conhecimento geográfico desses modelos serve a tarefas de predição quando o *prompt* traz contexto do OpenStreetMap.

Na descoberta de dados geoespaciais, a dificuldade de encontrar dados em geoportais é anterior aos modelos de linguagem. Hu et al. (2015) a atribuem, em parte, à

heterogeneidade dos metadados e propõem a harmonização de tópicos e a busca semântica em geoportais. Mais recentemente, Ning et al. (2025) propuseram um agente que escolhe a fonte de dados numa lista predefinida e gera o programa que a recupera, e Li et al. (2025) combinaram modelos de linguagem e um grafo de conhecimento construído a partir de cinco grandes portais para responder a perguntas de descoberta de dados.

3 DESENVOLVIMENTO

Este capítulo apresenta os requisitos da solução, o ambiente computacional e a sequência metodológica. A tradução da consulta em parâmetros foi desenvolvida em três versões sucessivas, que compartilham a API, a ferramenta de busca e o banco. A primeira versão oferece ao modelo uma única ferramenta de busca e faz uma chamada ao modelo por consulta. A segunda versão inclui nas descrições dessa ferramenta as convenções do catálogo e acrescenta uma ferramenta de recusa. A versão final acrescenta ferramentas auxiliares do domínio, que o modelo consulta antes de buscar, e um validador, que confere os parâmetros antes da busca e devolve ao modelo os erros encontrados, num laço de chamadas com limites fixos. As versões foram avaliadas em três conjuntos de consultas, o de referência, com 310 consultas, o de validação, com 1.929, e o de teste, com 945. A Seção 3.3.1 detalha as versões, e a Seção 3.3.5, os conjuntos. Os resultados de cada fase estão no Capítulo 4.

3.1 Requisitos

Os requisitos da solução têm três origens. A primeira são as demandas do cliente, apresentadas pelo Maj Diniz, do 1º CGEO, em 5 de fevereiro de 2026. A segunda é a análise do projeto `prototipo_busca_llm` (Seção 2.2), que fornece o esquema de busca a preservar e as limitações a superar. A terceira são as convenções que organizam o catálogo, a saber, a ET-PCDG (BRASIL. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016), a articulação sistemática das folhas do SCN (BRASIL, 1967), a divisão político-administrativa do IBGE e o Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) (BRASIL. Comissão Nacional de Cartografia (CONCAR), 2009). O manual de anotação (Seção 3.3.3.2) é a forma operacional dessas convenções e não constitui origem de requisito.

O Quadro 1 lista os requisitos funcionais (RF) e não funcionais (RNF), com a origem de cada um. A tradução da consulta em parâmetros está desdobrada por funcionalidade, do RF2 ao RF7, para que cada ferramenta descrita na Seção 4.2 se ligue a um requisito determinado.

Quadro 1 – Requisitos da solução e a sua origem

Id.	**Requisito**	**Origem**
RF1	Receber a consulta em linguagem natural e devolver os parâmetros, os produtos e a resposta ao usuário	cliente; `prototipo_busca_llm`
RF2	Reconhecer o lugar citado, seja a unidade da federação (UF), pelo nome ou pela sigla, seja o município, e restringir a busca pela interseção espacial com ele	`prototipo_busca_llm`; divisão político-administrativa do IBGE
RF3	Reconhecer a folha pedida pelo código MI ou INOM, em qualquer grafia, ou pelo nome	cliente; articulação sistemática das folhas
RF4	Reconhecer a escala em qualquer grafia e as expressões qualitativas de escala	cliente; escalas do catálogo
RF5	Reconhecer o tipo de produto, o centro de geoinformação e o projeto pelos nomes e apelidos usuais	`prototipo_busca_llm`; ET-PCDG
RF6	Resolver expressões de tempo em relação à data de execução, que o *prompt* informa ao modelo como data atual, e distinguir publicação de criação	`prototipo_busca_llm`; metadados do catálogo
RF7	Atender pedidos de ordenação e de quantidade, sem acrescentá-los quando não pedidos	`prototipo_busca_llm`
RF8	Recusar, com explicação ao usuário, o pedido que não é do acervo, sem recusar pedido legítimo	cliente; limitação 2 do `prototipo_busca_llm`
RF9	Tratar o pedido sem nenhum critério representável nos parâmetros	análise do `prototipo_busca_llm`
RF10	Informar a busca legítima sem resultado, com os critérios usados	cliente; `prototipo_busca_llm`
RF11	Registrar, por consulta, os parâmetros, as chamadas, os tempos e os erros	limitações 5 e 6 do `prototipo_busca_llm`
RNF1	Executar os modelos localmente	cliente
RNF2	Latência compatível com o uso interativo, sem limiar fixado *a priori*	cliente
RNF3	Compatibilidade com o esquema do `prototipo_busca_llm` e com os metadados do BDGEx (ET-PCDG e Perfil MGB)	`prototipo_busca_llm`; ET-PCDG
RNF4	Reprodutibilidade das avaliações	limitação 6 do `prototipo_busca_llm`
RNF5	Manutenibilidade, com testes automatizados e integração contínua	limitação 1 do `prototipo_busca_llm`
RNF6	Troca de modelo e de abordagem por configuração	limitação 3 do `prototipo_busca_llm`

Fonte: Elaborado pelos autores.

Nota: As limitações numeradas são as da Seção 2.2.2.

O RNF2 não tem limiar, porque a latência depende do *hardware* e da carga do ambiente de implantação, ambos fora do escopo do trabalho. Por esse motivo, a latência é reportada por ambiente de execução (Seção 3.2.2) e serve apenas ao desempate

(Seção 3.3.6.2). O atendimento de cada requisito é discutido na Seção 4.6.1.

3.2 Ambiente Computacional

Esta seção apresenta os componentes da solução e o *hardware*, justificado pelos requisitos dos modelos da Seção 2.1.3.

3.2.1 Componentes da Solução e Tecnologias

A Figura 3 mostra os componentes da solução e a tecnologia de cada um. O cliente envia a consulta à API, escrita em Python com o FastAPI (RAMÍREZ, 2018) e o Pydantic (COLVIN, 2019). O orquestrador da tradução, construído com o LangChain (CHASE, 2022), liga as ferramentas ao modelo, servido pelo Ollama (OLLAMA, 2023) no formato GGUF (GGML CONTRIBUTORS, 2023), e recolhe as chamadas de ferramenta. As ferramentas auxiliares, usadas só na versão final, consultam a lista de municípios do IBGE, o índice de folhas do BDGEx e uma tabela de expressões de tempo. A ferramenta de busca confere o formato dos parâmetros e monta a consulta SQL parametrizada sobre o PostgreSQL com PostGIS (POSTGIS DEVELOPMENT TEAM, 2001), executado em contêiner gerido pelo Docker Compose. Na versão final, o validador confere ainda os valores antes da busca e devolve ao modelo os erros encontrados. O *pipeline* de avaliação, fora do caminho do usuário, é usado para executar os conjuntos de consultas e pontuar as respostas.

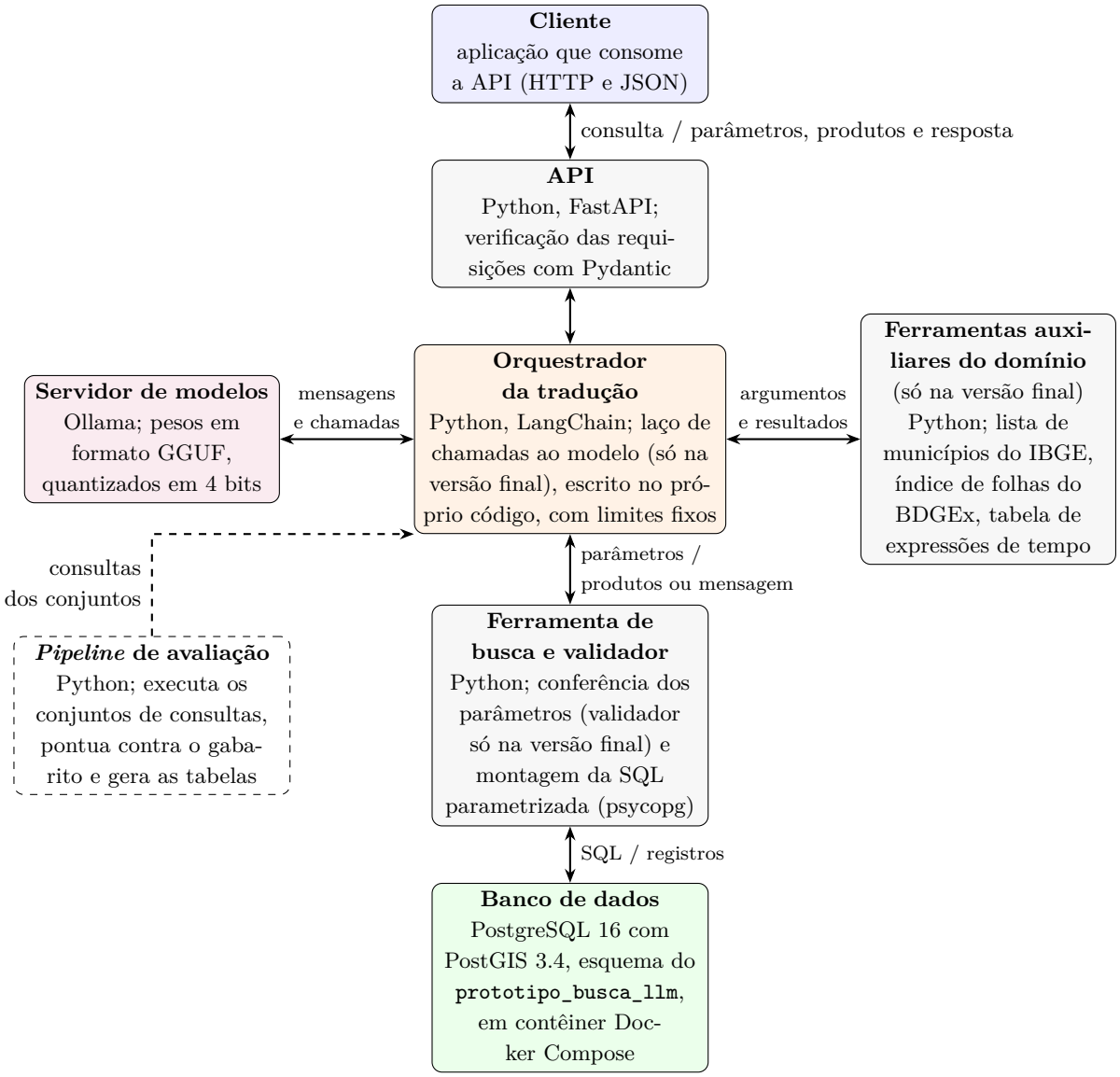


Figura 3 – Componentes da solução e tecnologia de cada um

Fonte: Elaborado pelos autores.

Do `prototipo_busca_llm` foram reaproveitados o esquema do banco, os doze parâmetros de busca e a lógica das consultas SQL, portada sem o filtro pelo retângulo do mapa e sem a paginação, próprios da interface gráfica, que ficou fora do escopo. Foram reescritos em Python a API, antes em Node.js, a tradução, que passou da saída estruturada com o Instructor (LIU, 2023) ao *tool calling*, e o acesso ao banco. Foram eliminados o dicionário de normalização e o *fallback* por expressões regulares, e cada falha do modelo passou a ser registrada e classificada (RF11). Foram criados as ferramentas auxiliares, o validador, os conjuntos de consultas e o *pipeline* de avaliação.

O modelo não escreve SQL. Ele apenas preenche os parâmetros tipados de uma ferramenta fixa, com os quais a ferramenta de busca monta a consulta parametrizada (Seção 2.3.2). Adotou-se essa formulação porque o texto do usuário não chega à SQL, o

que afasta a injeção (PEDRO et al., 2025), porque a geração de SQL tem acurácia ainda limitada em bancos reais (LI et al., 2023a) e fica exposta à ambiguidade das perguntas diante do esquema (LIU et al., 2025), e porque os enumerados da ET-PCDG permitem conferir cada valor antes da busca (RNF3). Em troca, a solução atende apenas às buscas que os doze parâmetros representam. Quanto à orquestração, o laço de chamadas da versão final é curto, tem limites fixos e foi escrito no próprio código do orquestrador, razão pela qual não se adotou o LangGraph (LANGCHAIN AI, 2024), que modela fluxos de agentes como grafos de estados.

A resposta ao usuário tem três desfechos, e em nenhum deles aparece produto que não esteja no banco:

1. no pedido fora do acervo, não há busca, e a resposta é a frase de recusa dirigida ao usuário;
2. na busca com produtos, a lista vem sempre do banco, e o modelo apenas a resume;
3. na busca legítima sem produto (RF10), a API devolve a lista vazia e uma mensagem fixa com os critérios usados, que sugere retirar ou ampliar algum deles, sem nova chamada ao modelo.

O modelo e a abordagem são escolhidos por configuração (RNF6). A API é empacotada em contêiner, e a sua configuração padrão é a versão final com o Gemma 4 E4B (Seção 4.6.4), com as versões anteriores ainda disponíveis. Os testes automatizados cobrem o *schema*, as consultas SQL, as medidas e a API sem depender do Ollama nem do banco (RNF5), e a integração contínua os executa a cada mudança. Uma alternativa mais leve, com SpatiaLite ou DuckDB e execução direta dos modelos pelo Hugging Face, sem orquestração de contêineres, fica como trabalho futuro (Seção 5.4).

Como o `prototipo_busca_llm` não traz os dados do acervo, a demonstração ponta a ponta usou primeiro um banco sintético (27 UFs, 14 municípios, 5 áreas de suprimento e 117 produtos), e o acervo real foi carregado depois (Seção 3.3.2). O código, os conjuntos de consultas, o manual de anotação completo e os registros das rodadas estão no repositório do projeto (CORRÊA; SANTANA, 2026).

3.2.2 *Hardware* e Requisitos dos Modelos

Os modelos locais da Tabela 2 ocupam de 2,5 GB a 7,1 GB em 4 bits, e o Phi-4 14B da solução do Maj Diniz, 9,1 GB, além da memória do contexto. Quando o total excede a memória da GPU (VRAM), parte do modelo é executada na CPU, e a latência aumenta. Esses requisitos justificam os três ambientes da Tabela 3.

A estação local (Tabela 3) representa o cenário de implantação modesta do RNF1 e do RNF2. Com 4 GB de VRAM, nenhum dos três modelos executados nela, o Gemma 4 E2B, o Gemma 4 E4B e o Qwen 3 4B, foi carregado inteiro na GPU, e a parcela carregada em cada caso é apresentada com os resultados da instalação (Seção 4.3).

Na máquina virtual do Google Colab com GPU NVIDIA T4, o Ollama foi executado na própria máquina, sem serviço de modelo em nuvem. Com cerca de 15 GB utilizáveis, a GPU T4 comporta inteiro cada modelo local, inclusive o Phi-4 14B, o que permite distinguir o efeito do modelo do efeito da divisão entre GPU e CPU. Por isso as rodadas de qualidade dos modelos locais nos três conjuntos foram feitas na GPU T4, e a latência foi medida em cada ambiente. A memória necessária a cada modelo inteiro não foi tratada como requisito medido, e a ocupação registrada para o Gemma 4 diferiu entre a GPU T4 e a estação local (Seção 4.3).

Os modelos maiores em nuvem, GPT-OSS 20B e 120B (OPENAI, 2025) e Qwen 3.8 27B (QWEN TEAM, 2026), servidos pelo Groq, excedem a VRAM da estação local e, executados fora dela, não atendem ao RNF1. A sua latência inclui a rede.

Tabela 3 – Ambientes de execução e uso de cada um

Ambiente	***Hardware***	**Uso no trabalho**	**Justificativa**
Estação local	Intel Core i7-12700H; 32 GB de memória principal; NVIDIA GeForce RTX 3050 Laptop com 4 GB de VRAM; Windows 11	latência e pares do teste A/B em parte do conjunto de referência; desenvolvimento da versão final	cenário de implantação modesta (RNF1, RNF2)
GPU T4 (Google Colab)	NVIDIA T4 com 16 GB nominais; Ollama na própria máquina virtual	rodadas de qualidade dos modelos locais nos três conjuntos; latência na GPU T4	comporta cada modelo local inteiro, sem divisão entre GPU e CPU
Groq (nuvem)	*hardware* do provedor, não controlado	primeira versão com os modelos maiores no conjunto de referência, incluídos os pares do teste A/B	só para situar os modelos locais; não atende ao RNF1

Fonte: Elaborada pelos autores.

3.3 Sequência Metodológica

A sequência metodológica segue as cinco fases previstas para o trabalho, que são a extração dos dados, a modelagem das funcionalidades, a instalação dos modelos, a validação dos modelos e a avaliação de performance. A Figura 4 apresenta essa sequência como mapa de processos na notação BPMN (*Business Process Model and Notation*) (OBJECT MANAGEMENT GROUP, 2014) e mostra a iteração como ocorreu. A primeira versão foi

modelada, os modelos foram instalados e validados no conjunto de referência, e só então os metadados do BDGEx foram extraídos para montar os conjuntos de validação e de teste. A segunda versão e a versão final foram modeladas em rodadas de correção e, congelada a versão final, a performance foi avaliada. Por fim, foi feita a análise complementar no acervo real. As tarefas destacadas em azul tiveram, além do apoio ao desenvolvimento do código, apoio de agentes de inteligência artificial (IA) (Seção 3.3.7). Depois da visão geral das versões, as subseções seguem as fases, isto é, a extração dos dados (tarefas 1 e 5), a modelagem das funcionalidades (tarefas 2 e 6), a instalação dos modelos (tarefa 3), a validação dos modelos (tarefas 4 e 5) e a avaliação de performance (tarefas 7 e 8).

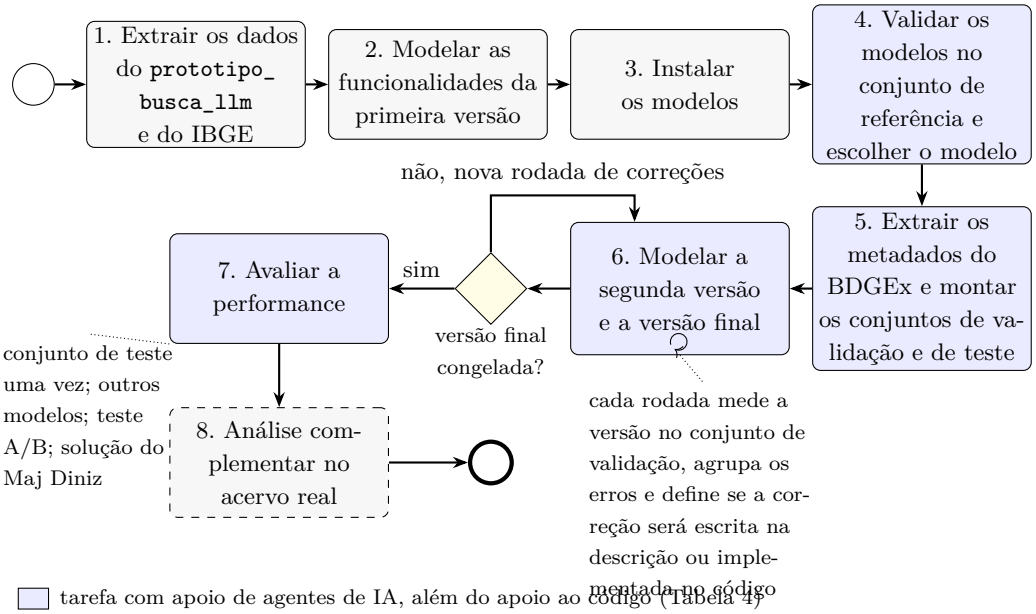


Figura 4 – Sequência metodológica em notação BPMN

Fonte: Elaborado pelos autores, na notação de Object Management Group (2014).

3.3.1 Visão Geral das Três Versões

As três versões diferem apenas na tradução da consulta em parâmetros, e a API, a ferramenta de busca e o banco são comuns às três. Foram desenvolvidas em sequência, a primeira a partir da solução do Maj Diniz e cada uma das seguintes a partir dos erros da anterior, e cada versão foi testada num conjunto de consultas que não serviu para desenvolvê-la.

A primeira versão oferece ao modelo uma única ferramenta de busca, com os doze parâmetros do `prototipo_busca_llm`, e faz uma chamada ao modelo por consulta, sem novas tentativas. Difere da solução do Maj Diniz por usar *tool calling* e por dispensar o dicionário de normalização e os exemplos resolvidos no *prompt*. Para recusar um pedido, o modelo não chama a ferramenta. Essa versão foi usada no conjunto de referência para

escolher o modelo e para a comparação com a saída estruturada e com a solução do Maj Diniz.

A segunda versão mantém a ferramenta de busca e uma única chamada ao modelo por consulta, inclui nas descrições dos parâmetros as convenções do manual de anotação (Seção 3.3.3.2) e acrescenta uma ferramenta de recusa. Foi desenhada a partir dos erros da primeira versão no conjunto de referência e testada no conjunto de validação.

A versão final revisa as descrições e acrescenta quatro ferramentas auxiliares do domínio, que identificam nomes, normalizam códigos e escalas e resolvem expressões de tempo. Acrescenta também um pedido de esclarecimento, a verificação dos parâmetros com retorno ao modelo e a recusa contestada, pela qual a primeira recusa de uma consulta que cita um critério do catálogo volta ao modelo para confirmação, num laço de chamadas com limites fixos. Foi desenvolvida nos conjuntos de referência e de validação, congelada e, em seguida, testada uma única vez no conjunto de teste, com o procedimento descrito na Seção 3.3.6.4.

A construção e o papel de cada conjunto estão na Seção 3.3.5, e o detalhe de cada versão, com a motivação de cada ferramenta, na Seção 4.2.

3.3.2 Extração dos Dados

A extração reuniu dados de três fontes, cujas estatísticas estão na Seção 4.1. Do `prototipo_busca_llm` (1º CENTRO DE GEOINFORMAÇÃO, 2025) vieram os 22 casos de teste e o esquema do banco, com quatro tabelas. Do IBGE vieram a lista de municípios e as malhas das UFs e dos municípios. Do BDGEx foram coletados, pelo serviço CSW (OPEN GEOSPATIAL CONSORTIUM, 2007), em 28/09/2026, apenas os metadados públicos, sem baixar nenhum produto. Desses metadados foram extraídos os nomes de folha, os códigos, as escalas, os tipos de produto, os CGEOs e os anos usados nos conjuntos de validação e de teste, além do índice de folhas que as ferramentas da versão final consultam.

Para a análise complementar, os metadados foram carregados num PostgreSQL com PostGIS, no esquema do `prototipo_busca_llm`, junto das malhas do IBGE. Como o CSW não traz a geometria da folha nem campos de tipo, de escala e de projeto, a geometria de cada registro é a caixa envolvente informada, o tipo e a escala foram inferidos do título e do formato, e as áreas de suprimento, sem polígono oficial, foram aproximadas pela união das caixas dos registros de cada CGEO.

3.3.3 Modelagem das Funcionalidades

A modelagem definiu as perguntas e as exceções que o sistema deve resolver, fixou num manual a resposta correta de cada consulta e organizou o desenvolvimento em ciclos

de correção.

3.3.3.1 Tipos de Consulta e Exceções

O sistema deve resolver os tipos de consulta listados a seguir, que podem se combinar numa mesma consulta:

- simples, como “cartas de São Paulo”;
- compostas, como “ortoimagens de Manaus em 1:50.000”;
- por código MI ou INOM, como “INOM SG-22-X-A”;
- com tempo relativo, como “nos últimos 3 meses”;
- com ordenação e quantidade, como “as 3 cartas mais antigas”;
- informais ou ambíguas, com siglas e grafias sem acento.

Os tipos de consulta correspondem aos requisitos RF2 a RF7. Há também cinco exceções. As consultas fora do domínio, como “cartas de Marte”, pedem o que o acervo não tem e devem ser recusadas (RF8). As consultas sem critério, como “cartas do Nordeste”, não têm nenhum critério representável nos parâmetros e admitem a busca sem filtros ou a não busca (RF9). A busca legítima sem resultado deve ser informada ao usuário (RF10). O valor que os parâmetros não representam, como “escala 1:10.000.000”, não tem comportamento correto determinado pelo texto e é um caso observacional, fora da pontuação. O nome ambíguo, como São Paulo, estado ou capital, admite mais de uma leitura (RF2). As consultas que não são fora do domínio nem sem critério são chamadas consultas do domínio.

3.3.3.2 Manual de Anotação

O manual de anotação é o documento que define a resposta correta de cada consulta, chamada gabarito. Foi escrito pelos autores antes de se anotarem as consultas, com base no gabarito original dos casos de teste do `prototipo_busca_llm` e nas convenções do catálogo, e tem seis princípios (evidência explícita no texto, forma canônica, leituras múltiplas explícitas, tratamento das consultas fora do domínio, consistência e casos observacionais) e regras para cada um dos doze parâmetros, entre elas as regras de tempo relativo, cada uma identificada por um nome e resolvida na data de execução.

Parte das consultas admite mais de uma resposta correta, como “últimos 3 meses” (noventa dias ou três meses de calendário) ou “as mais recentes” (por data de publicação ou de criação). O gabarito registra todas as leituras aceitas por meio de três mecanismos, a saber, o valor alternativo (mais de um valor aceito num parâmetro), o campo opcional

(parâmetro que pode faltar) e a leitura estrutural alternativa (o mesmo trecho em parâmetros diferentes), cada um justificado por uma regra. Entre as leituras aceitas, uma é marcada como leitura preferencial. Fora dessas leituras, a comparação é estrita.

O manual foi estendido aos conjuntos de validação e de teste sem alterar nenhuma anotação do conjunto de referência, e o texto completo está no repositório do projeto (CORRÊA; SANTANA, 2026). O manual nunca foi passado ao modelo, pois a sua função é definir o gabarito. Parte das suas convenções foi escrita nas descrições da ferramenta na segunda versão, e parte foi implementada em código, nas ferramentas e no validador da versão final.

3.3.3.3 Ciclo de Desenvolvimento

A segunda versão e a versão final resultaram de rodadas de um mesmo ciclo. A versão é medida num conjunto de desenvolvimento (o de referência ou o de validação), os erros são agrupados por tipo, e define-se o lugar de cada correção. A correção é implementada no código quando a regra é determinística e verificável na própria consulta, como a forma canônica, a tabela de datas ou as listas de UFs, municípios e folhas, e é escrita nas descrições e no *prompt* quando se trata de orientar o uso das ferramentas. Na versão final, confere-se ainda que o validador não acusa nenhuma resposta certa do gabarito. Os ciclos da versão final foram registrados sobre uma amostra estratificada de 160 consultas do conjunto de validação, com o Gemma 4 E4B na estação local, e agentes de IA foram empregados para classificar os erros e propor causas (Seção 3.3.7). O conjunto de teste ficou fora do ciclo (Seção 3.3.5.3).

3.3.4 Instalação dos Modelos

Os modelos locais foram selecionados por terem suporte nativo a *tool calling* no Ollama, conferido pela etiqueta do catálogo oficial (OLLAMA, 2023), versões quantizadas em 4 bits executáveis em estações modestas, com parte do modelo na CPU quando a VRAM é insuficiente, e lançamento entre 2024 e 2026, e por cobrirem três organizações, a Alibaba, o Google e a Mistral AI, com dois portes da família Gemma 4. São o Qwen 3 4B (QWEN TEAM, 2025a), o Gemma 4 E2B e o Gemma 4 E4B (GEMMA TEAM, 2026), além do Mistral Nemo 12B (MISTRAL AI, 2024) (Tabela 2). O Phi-4 14B (ABDIN et al., 2024) foi usado apenas na solução do Maj Diniz.

Todos foram instalados pelo Ollama com as etiquetas oficiais e executados com temperatura zero (o modelo escolhe sempre o *token* mais provável), limite de 1.024 *tokens* de saída e contexto de 4.096 *tokens* na primeira e na segunda versão e de 8.192 na versão final, porque o *prompt* desta, com várias ferramentas, se aproxima de 4.096 *tokens*. Cada consulta é enviada sem conversas anteriores, e todos os modelos recebem, em cada versão, o mesmo *prompt* de sistema, com a data de execução.

O raciocínio interno foi desligado por parâmetro, para que as latências fossem comparáveis, e o ganho possível com ele não foi medido. Na versão padrão do Qwen 3 4B, o raciocínio não podia ser desligado e, ao esgotar o limite de saída, impedia a chamada de ferramenta. Por isso, adotou-se a versão oficial sem raciocínio, Qwen3-4B-Instruct-2507, do mesmo porte. No Groq, usou-se o menor esforço de raciocínio aceito por cada modelo (GROQ, 2026), isto é, nenhum raciocínio no Qwen 3.8 27B e o esforço baixo nos GPT-OSS 20B e 120B, porque a família GPT-OSS não permite desligá-lo. O resultado da instalação está na Seção 4.3.

3.3.5 Validação dos Modelos

Nenhum conjunto de consultas foi usado para treinar ou ajustar os pesos de um modelo. Os conjuntos de referência e de validação orientaram o desenvolvimento das descrições, das ferramentas e do validador, e cada consulta é enviada ao modelo sozinha, sem exemplos resolvidos.

3.3.5.1 Conjunto de Referência

O conjunto de referência reúne 310 consultas de três origens disjuntas, cujas quantidades se somam. As consultas do protótipo são os 22 casos de teste do `prototipo_busca_llm`, rastreados até a linha de origem no arquivo do protótipo, com a anotação original como leitura preferencial. As 40 consultas redigidas pelos autores cobrem lacunas, como as consultas simples, ausentes no protótipo, e casos de fronteira. As 248 consultas geradas por modelos de frase combinam frases do português com listas fechadas do domínio (UFs, municípios, escalas, tipos, CGEOs, projetos, códigos, expressões de tempo e ordenações), com o gabarito conhecido por construção e geração determinística. As 62 consultas do protótipo e dos autores formam o *subconjunto manual*.

O gabarito passou por uma auditoria em três passos. No primeiro, foram conferidas por checagens automáticas a unicidade, a validade dos valores, a resolução das regras de tempo em datas diversas, a rastreabilidade e a consistência, com o apoio de um anotador por regras escrito pelos autores. Um agente de IA de uma família que não está entre as avaliadas anotou as consultas de forma independente e às cegas, em dez blocos, só com o manual, a definição da ferramenta, a data de execução e o bloco. Os autores adjudicaram cada divergência e registraram a decisão. Um dos autores conferiu ainda à mão uma amostra estratificada de 40 consultas.

3.3.5.2 Conjuntos de Validação e de Teste

No conjunto de referência, as conclusões podem depender da regularidade das consultas geradas, a recusa é medida em poucas consultas fora do domínio, e o resultado pode refletir mais as descrições e o *prompt* da versão avaliada que a abordagem. Por isso

foram construídos dois conjuntos novos, segundo o mesmo procedimento. Dos metadados do BDGEx e da lista do IBGE foram sorteados alvos, isto é, especificações de consultas a redigir (1.950 para o conjunto de validação e 975 para o de teste), distribuídos em oito famílias que seguem os tipos de consulta e as exceções da Seção 3.3.3.1, a saber, as consultas simples, as compostas, as por código, as de tempo relativo, as de ordenação, as de leituras múltiplas ou ambíguas, as sem critério e as fora do domínio. Cada alvo fixa, antes de o texto existir, o gabarito segundo o manual, a forma como cada valor deve aparecer (como "25k" ou "3o cgeo"), o que a consulta não pode dizer, um registro de linguagem e uma persona. Agentes de IA em sessões independentes redigiram as consultas sem ver o gabarito, e as repetições foram eliminadas por checagens automáticas. Outros agentes anotaram as consultas de forma independente, embaralhadas e sem o gabarito, e só foi aceita a consulta compatível com o gabarito nos dois sentidos (a leitura do anotador está entre as aceitas, e a preferencial está entre as que o anotador aceita). Uma amostra de 20% recebeu uma segunda anotação, fora desse filtro.

O conjunto de teste tem cerca de metade do tamanho do conjunto de validação, foi sorteado com outra semente aleatória e redigido em outras sessões de agentes, e nenhuma consulta dele foi lida no desenvolvimento. Os relatórios dos anotadores citaram trechos de algumas dezenas delas, e nada da versão final foi alterado a partir deles, nem há exemplo das descrições ou dos *prompts* que coincida com uma consulta inteira desse conjunto. Nos dois conjuntos, porém, os nomes e os códigos foram sorteados das mesmas listas que as ferramentas consultam, e o alcance das ferramentas fora dessas listas não foi medido.

3.3.5.3 Papéis dos Conjuntos e Congelamento

O conjunto de referência serviu à escolha do modelo, às comparações da primeira versão e ao desenvolvimento, e o de validação, ao teste da segunda versão e ao desenvolvimento da versão final. Com o de teste, a versão final foi medida uma única vez, porque um conjunto de teste usado para ajustar o sistema deixa de ser de teste (DWORK et al., 2015). Um número é dito *dentro da amostra* quando foi medido num conjunto usado no desenvolvimento da própria versão medida, como a versão final no conjunto de referência, e é reportado apenas como registro, sem valor de teste. Uma análise é dita *exploratória* quando não estava no plano registrado antes da rodada de teste e não alterou a versão final.

Para congelar a versão final antes da rodada de teste, calculou-se o resumo criptográfico (*hash* SHA-256) do texto dos *prompts*, das definições das ferramentas e do código e dos dados das ferramentas. O resumo é uma sequência curta calculada a partir do conteúdo, e qualquer alteração no conteúdo o modifica. A rodada de teste só foi executada depois de se conferir que os resumos eram iguais aos da versão congelada, e o plano de análise foi registrado no repositório antes dela. Para a reprodutibilidade (RNF4), cada

rodada registra o modelo, as versões do *software*, a configuração, a data de execução e o *hardware*, e a pontuação é recalculada contra o gabarito vigente sem nova execução.

3.3.6 Avaliação de Performance

A performance foi avaliada isoladamente e em comparação com a solução do Maj Diniz, com os conceitos estatísticos da Seção 2.3.4.

3.3.6.1 Medidas

A medida principal é a acurácia por consulta, ou acerto por parâmetros. A consulta está certa quando os parâmetros coincidem com uma leitura aceita, sem parâmetro a mais nem a menos, ou, nas consultas em que o correto é não buscar, quando não há busca. Os enumerados exigem igualdade exata, os nomes livres são comparados após normalizar caixa, acentos e pontuação, e os períodos exigem os mesmos limites. A acurácia é acompanhada do intervalo de confiança (IC) de 95% de Wilson (WILSON, 1927). No conjunto de referência, as rodadas da primeira versão com *tool calling* nos modelos locais foram executadas três vezes na mesma sessão, e as demais rodadas, em todos os conjuntos, uma vez. Nessas rodadas repetidas, as tabelas trazem a média das três repetições, e os testes estatísticos usam o resultado majoritário das três, porque repetições da mesma sessão quase sempre coincidem e contá-las como independentes exageraria a significância.

Reportam-se também o F1 dos campos, ponderado pelas ocorrências, a acurácia no domínio e a recusa como classificação, cujo positivo é a consulta fora do domínio, com precisão, *recall*, F1 da recusa e falsa recusa (não buscar numa consulta do domínio). Nas consultas sem critério, qualquer não busca conta como acerto. A latência é medida depois de uma consulta de aquecimento e reportada pela mediana e pelo percentil 95 por ambiente. Ela abrange o intervalo do envio da consulta à resposta final, inclui as chamadas ao modelo, as ferramentas e o validador e exclui o tempo de execução da SQL.

Para verificar se os erros contados se deviam ao modelo ou ao gabarito, sortearam-se 160 execuções erradas da primeira e da segunda versão no conjunto de validação, 40 por configuração de *tool calling* e de saída estruturada, estratificadas por família de consulta, e um agente de IA as julgou segundo o manual. Essa auditoria dos erros é distinta da auditoria do gabarito descrita na Seção 3.3.5.1.

3.3.6.2 Regra de Decisão

As medidas não são combinadas num escore único, e a acurácia por consulta é o critério principal. Duas configurações são comparadas nas mesmas consultas pelo teste de McNemar exato e bicaudal (MCNEMAR, 1947), com uma observação por consulta. Há diferença quando o p-valor é menor que 0,05; do contrário, há empate, que não de-

monstra equivalência, porque diferenças de alguns pontos percentuais (p.p.) podem passar sem detecção. O *bootstrap* pareado (EFRON; TIBSHIRANI, 1993) estima o tamanho da diferença, e a sobreposição de intervalos de Wilson não é usada como teste. Entre configurações empatadas, a preferência segue, sem teste, o F1 dos campos, a recusa e o custo (latência, chamadas ao modelo e parcela do modelo na GPU), critérios que não revertem uma diferença significativa de acurácia.

O plano registrado antes da rodada de teste fixou a medida, os testes, a comparação principal, os passos da versão final e a separação entre a resposta final e a primeira decisão, que é a primeira busca ou recusa anterior a qualquer retorno do validador. O nível de significância e a ordem dos desempates foram escritos depois das rodadas, e a correção de Holm (HOLM, 1979), aplicada às nove comparações planejadas do conjunto de teste, não estava no plano.

3.3.6.3 Comparação com a Solução do Maj Diniz

A solução do Maj Diniz foi reproduzida em Python com o Phi-4 14B, o dicionário de 159 pares, o *prompt* original em inglês com dez exemplos resolvidos, o *schema* com campo de justificativa, até três novas tentativas e o *fallback* por expressões regulares. As diferenças deliberadas são a temperatura zero (0,6 no protótipo), a exclusão da ordenação padrão que o protótipo acrescenta a toda busca e a ausência de conferência dos estados e municípios no banco. A saída é traduzida para os parâmetros pela correspondência usada no gabarito das consultas do protótipo. A solução foi executada uma vez, só no conjunto de referência, e comparada com a primeira versão nos quatro modelos locais e com a saída estruturada da primeira versão (Seção 3.3.6.5).

3.3.6.4 Passos da Versão Final e Controle

No conjunto de teste, com o Gemma 4 E4B, a primeira e a segunda versão são seguidas de três passos cumulativos, aplicados em ordem fixa e designados como "só com as descrições revisadas", "com as ferramentas auxiliares" e "com a verificação dos parâmetros". O primeiro usa só as descrições revisadas, com uma chamada ao modelo. O segundo acrescenta as ferramentas auxiliares, o pedido de esclarecimento (respondido por um usuário simulado de resposta fixa, que diz não ter outros detalhes), o pedido para responder chamando uma ferramenta, enviado uma vez ao modelo quando a sua resposta não contém chamada, e o laço de chamadas. O terceiro acrescenta a verificação dos parâmetros e a recusa contestada, formando a versão final. Cada diferença mede o efeito do passo, mantidos os anteriores. Nos três passos, a chamada de ferramenta que o modelo escreve no texto da resposta, em vez de emiti-la pelo canal de ferramentas, é interpretada como chamada (Seção 4.2.3).

A versão final é comparada com um controle, a saída estruturada com as mesmas

descrições revisadas e a mesma verificação dos parâmetros. Como a saída estruturada emite um único objeto e não consulta ferramentas durante a resposta (Seção 2.3.2), o validador e a recusa contestada são aplicados a ele como novas tentativas, no máximo duas, cada uma depois de um objeto completo. O controle não recebe as ferramentas auxiliares, o usuário simulado, o pedido de resposta por ferramenta, a leitura das chamadas escritas no texto nem a instrução do *prompt* segundo a qual um lugar que o modelo não reconhece não justifica a recusa (Seção 4.2.3.2). Se não houver diferença significativa entre o controle e a versão final, o ganho deve-se às descrições e à verificação. Se o controle tiver acurácia menor, a diferença mede o mecanismo de *tool calling* junto com essas peças, sem separá-los, porque o controle que os separaria, com a saída estruturada recebendo o retorno das ferramentas auxiliares, não foi executado. Como as ferramentas e o validador codificam o manual que define o gabarito, inclusive a tabela de datas, a primeira decisão é analisada à parte da resposta final.

Conforme um plano registrado antes dessas rodadas, a comparação principal, o ganho sobre a primeira versão e a comparação entre *tool calling* e saída estruturada na primeira versão foram repetidos com o Gemma 4 E2B e o Qwen 3 4B, na mesma versão final congelada. Essas rodadas usaram uma revisão posterior do código e o Ollama 0.35.0, contra 0.34.4 nas do Gemma 4 E4B, e por isso as latências dos dois grupos não são comparáveis. O plano não previa comparar os modelos entre si.

3.3.6.5 Teste A/B

Para medir o ganho do *tool calling* sobre a saída estruturada (Seção 1.2.3), as rodadas das duas abordagens foram reunidas num teste A/B. No Modo A (*tool calling*), as ferramentas definidas em JSON Schema são ligadas ao modelo pelo LangChain, e o modelo escolhe, pelo canal de ferramentas do Ollama ou do Groq, se chama cada uma e com quais argumentos. No Modo B (saída estruturada), o JSON Schema dos mesmos parâmetros é incluído no *prompt*, e a resposta em modo JSON passa pela mesma classe Pydantic, que confere o formato sem restringir a geração. Na primeira versão, o Modo B não oferece meio de recusa, e a partir da segunda um campo próprio permite a recusa. A geração restrita ao *schema* foi descartada, porque, com ela, os modelos encerravam o intervalo de datas logo depois do primeiro limite, sem o segundo, artefato que penalizaria o Modo B e é compatível com a observação de Tam et al. (2024) de que restrições de formato alteram o desempenho dos modelos.

Cada par tem o mesmo modelo, as mesmas consultas, a mesma versão e o mesmo ambiente. São 24 pares. Em 19 deles, a versão comparada não foi desenvolvida no conjunto em que foi medida, e nos demais o resultado é dentro da amostra. O ganho em acurácia segue a regra de decisão, e o ganho em tempo é avaliado pela latência e pelo número de chamadas ao modelo, comparados só entre rodadas com a mesma GPU, a mesma versão

do Ollama e a mesma parcela do modelo na GPU, o que exclui o Groq.

3.3.6.6 Avaliação pela Execução no Acervo Real

O princípio da avaliação pela execução (Seção 2.3.4), que compara o resultado da consulta, e não o seu texto, foi aplicado numa análise complementar, sem novas rodadas de modelos. No acervo carregado (Seção 3.3.2), cada leitura aceita das consultas do conjunto de teste em que a resposta correta é buscar foi expandida em buscas concretas (cada valor alternativo; cada campo opcional com e sem ele), com as datas resolvidas em 24/09/2026, a data de execução das rodadas do conjunto de teste, e a busca emitida por cada configuração nas rodadas existentes foi executada com a mesma SQL. O acerto por resultado exige o mesmo conjunto completo de cartas de alguma busca aceita ou, numa consulta sem critério, a não busca. Como muitas buscas do gabarito devolvem zero cartas, o acerto por resultado é calculado também apenas nas consultas cujo gabarito tem cartas. A análise é limitada porque o CSW não traz o projeto, grava uma só data por registro e não tem polígono oficial da área de suprimento, e porque o tipo e a escala foram inferidos do título.

3.3.7 Uso de Ferramentas de Inteligência Artificial

A família de modelos usada como apoio, o Claude, da Anthropic, não está entre os modelos avaliados. A Tabela 4 discrimina o uso por fase. O código foi desenvolvido com apoio do Claude Code. Em nenhum processo a IA definiu o gabarito. No conjunto de referência, ele foi definido pelos autores segundo o manual, e a anotação por agente serviu para medir a concordância e apontar divergências; nos conjuntos de validação e de teste, foi fixado por construção, antes da redação. A auditoria dos erros (Seção 3.3.6.1) coube a um único agente, sem conferência humana, e uma só família de modelos redigiu, anotou e julgou (Seção 4.6.5).

Tabela 4 – Uso de ferramentas de inteligência artificial por fase da sequência metodológica

Fase	**Ferramenta**	**Finalidade**	**Controle do resultado**
Todas as fases (código)	Claude Code, com os modelos Claude Opus 4.6 e Claude Opus 5.5	desenvolvimento do código da coleta, do *backend*, das ferramentas, do validador, do *pipeline* de avaliação e dos testes	testes automatizados, revisão pelos autores e repositório público
Extração dos dados (coleta, tarefas 1 e 5) e instalação dos modelos (tarefa 3)	nenhuma além do código	não se aplica	não se aplica
Modelagem das funcionalidades (tarefas 2 e 6)	agentes Claude	classificar os erros das rodadas de desenvolvimento e propor causas	números recalculados por código; causas não testadas registradas como hipóteses
Validação dos modelos (tarefa 4 e montagem dos conjuntos na tarefa 5)	agentes Claude	anotar às cegas o conjunto de referência; redigir e anotar às cegas os conjuntos de validação e de teste	concordância medida; divergências adjudicadas pelos autores; checagens automáticas; filtro de compatibilidade; segunda anotação de uma amostra; conferência humana de 40 consultas do conjunto de referência
Avaliação de performance (tarefas 7 e 8)	agentes Claude	julgar 160 execuções erradas do conjunto de validação na auditoria dos erros (um único agente, sem conferência humana); classificar os erros restantes; conferir as análises	números recalculados por código

Fonte: Elaborada pelos autores. Os agentes usaram os modelos Claude Opus 5.5 e Claude Fable 5.1 (Anthropic).

4 RESULTADOS

Este capítulo segue a ordem das fases da sequência metodológica (Figura 4). Apresenta os dados extraídos do acervo, as funcionalidades da busca, a instalação e a validação dos modelos, a avaliação de performance, a discussão dos resultados e os entregáveis.

4.1 Extração dos Dados

A coleta dos metadados públicos do BDGEx pelo serviço CSW, feita em 28/09/2026 pelo procedimento da Seção 3.3.2, produziu o acervo sobre o qual a busca foi executada na avaliação da Seção 4.5.4. O catálogo declara 31.190 registros, mas a coleta trouxe 21.229 registros distintos, porque o servidor repetiu páginas durante a paginação e 9.961 cópias idênticas foram descartadas; são esses registros distintos que formam o acervo coletado. Não se verificou se o número declarado conta registros distintos; o mais provável é que parte do catálogo não tenha sido coletada, e por isso as contagens de folhas homônimas e de edições apresentadas a seguir são limites inferiores.

A Tabela 5 resume a composição do acervo coletado. A maior parte dos registros está nas escalas 1:25.000 e 1:50.000, e as cartas topográficas matriciais e vetoriais predominam entre os tipos de produto. Quase todos os registros trazem o código MI e o INOM da folha. O CSW não tem campo de tipo de produto nem de escala, e os dois foram inferidos do título e do formato de cada registro por regras heurísticas, com 1.052 registros sem tipo reconhecido e 165 sem escala reconhecida. Dos valores que o enumerado do catálogo prevê, 5 tipos de produto e 2 escalas não aparecem no acervo coletado. Do total, 3.711 registros vêm de produtor que não é um CGEO.

Tabela 5 – Composição do acervo do BDGEx coletado pelo CSW

Item	**Registros**	**% dos distintos**
Coleta		
Registros declarados pelo catálogo	31.190	—
Registros distintos coletados	21.229	—
Cópias idênticas descartadas	9.961	—
Escala		
1:25.000	8.534	40,2%
1:50.000	8.642	40,7%
1:100.000	3.150	14,8%
1:250.000	698	3,3%
1:2.000	33	0,2%
1:10.000	7	—
Sem escala reconhecida	165	0,8%
Tipo de produto		
SCN Carta Topográfica Matricial	10.536	49,6%
SCN Carta Topográfica Vetorial	6.720	31,7%
MDS (modelo digital de superfície)	1.462	6,9%
SCN Carta Ortoimagem	1.459	6,9%
Sem tipo reconhecido	1.052	5,0%
Códigos		
Registros com MI	20.928	98,6%
Registros com INOM	21.005	98,9%
Códigos MI distintos	10.634	—
INOM distintos	10.971	—

Fonte: Elaborada pelos autores a partir dos metadados públicos do BDGEx, coletados em 28/09/2026.

Nota: Tipo e escala inferidos do título e do formato de cada registro, porque o CSW não tem esses campos.

Além dos metadados, a versão final da busca depende de dois conjuntos de dados que as suas ferramentas consultam, a lista de municípios do IBGE, com 5.571 municípios, e o índice de folhas do BDGEx, com os códigos e os nomes das folhas do acervo. Para a avaliação pela execução no acervo real, foram carregadas também as malhas territoriais do IBGE, com 27 UFs e 5.570 municípios com polígono (1 município da lista ficou sem polígono), e as 5 áreas de suprimento aproximadas (Seção 3.3.2).

A Tabela 6 lista os principais vícios do acervo, como as folhas homônimas, os municípios de nome ambíguo e a mesma folha em vários registros, com a frequência de cada um e a funcionalidade da Seção 4.2 que se relaciona com ele ou a limitação que permanece.

Esses vícios foram observados na avaliação pela execução no acervo real, feita depois do desenvolvimento da busca, e nenhuma funcionalidade foi desenhada a partir deles. As frequências por pedido foram medidas nos pedidos do conjunto de teste executados no acervo, e as demais foram contadas no acervo coletado; todas descrevem o acervo, e não o acerto de algum modelo. Até essa avaliação, a busca só tinha sido executada no banco sintético da Seção 3.2.1, que não reproduz nenhum desses vícios.

Tabela 6 – Vícios do acervo e funcionalidade ou limitação relacionada a cada um

Vício	**Exemplo**	**Frequência**	**Funcionalidade ou limitação**
Folhas homônimas em lugares distintos	Terra Nova em três folhas de lugares distintos	12 dos 111 pedidos por nome de folha	a identificação de nomes (Seção 4.2.3.2) informa as escalas em que a folha existe; a desambiguação pela UF fica como trabalho futuro
Municípios homônimos ou contidos em outro nome	"Santa Maria" corresponde a 17 municípios	33 dos 135 pedidos por município	a identificação de nomes devolve o município com a UF; o filtro por trecho do nome, herdado do `prototipo_busca_llm`, continua
Mesma folha em vários registros (produtos, edições e duplicatas)	um código com até 5 registros	95 dos 130 pedidos por código; 890 grupos de duplicatas com 1.781 registros	a ordem por data mostra primeiro a edição mais recente (em 98 pedidos por código)
MI escrito com e sem zero à esquerda	"0757-4" e "757-4"	500 códigos MI	limitação; a normalização de códigos (Seção 4.2.3.3) mantém os algarismos como escritos, e a busca por igualdade não une as duas grafias (trabalho futuro)
Nomes que diferem só por acento ou caixa	"Vila Propício" e "Vila Propicio"	281 INOM	limitação; a busca por palavra-chave é sensível a acento
Campos ausentes ou aproximados no CSW	projeto vazio em todos os registros; uma só data por registro; área de suprimento sem polígono oficial	todos os registros	limitação declarada na avaliação pela execução (Seção 4.5.4)

Fonte: Elaborada pelos autores. Frequências por pedido medidas nos pedidos do conjunto de teste executados no acervo real; as demais, contadas no acervo coletado.

4.2 Modelagem das Funcionalidades

As três versões da tradução estão resumidas na Seção 3.3.1. Esta seção detalha cada desenvolvimento na ordem em que ocorreu, com o erro ou a ambiguidade que motivou cada peça, o conceito da Engenharia Cartográfica que ela codifica e o requisito do Quadro 1 que atende. Os números que motivam cada peça vêm dos conjuntos de referência e de validação, usados no desenvolvimento; os do acervo real, citados na Seção 4.2.4, só descrevem o filtro espacial, e os efeitos no conjunto de teste estão na Seção 4.5. A Tabela 7 resume a seção, e o código está no repositório do projeto (CORRÊA; SANTANA, 2026). Como mostra a Figura 4, a segunda versão e a versão final foram modeladas depois da instalação e da validação dos modelos (Seções 4.3 e 4.4). Por isso, esta seção usa os erros observados nos conjuntos de referência e de validação apenas como motivação de cada peça, e as comparações entre abordagens e as do conjunto de teste estão na Seção 4.5.

4.2.1 Primeira Versão e a Ferramenta de Busca

A primeira versão expõe ao modelo uma única ferramenta, `buscar_catalogo`, com os doze parâmetros do *schema* do `prototipo_busca_llm`, e faz uma chamada ao modelo por consulta. Quando o pedido não é do acervo, o modelo recusa deixando de chamar a ferramenta e respondendo em uma frase. O *prompt* de sistema informa a data de execução e não traz exemplos resolvidos nem dicionário de termos, e as descrições dos parâmetros substituem os 159 pares do dicionário de normalização do `prototipo_busca_llm`.

O *schema* foi mantido pelas razões da Seção 3.2.1. Os enumerados foram recortados para os valores do catálogo, com nove tipos de produto e nove projetos, contra 30 e 15 no `prototipo_busca_llm`, e os nomes foram escritos conforme a ET-PCDG (BRASIL. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016).

Os parâmetros agrupam os conceitos da cartografia sistemática que uma consulta pode citar. A folha é identificada, no parâmetro de palavra-chave, pelo código MI, pelo INOM ou pelo nome (Seção 4.2.3.3). A escala segue o enumerado do catálogo, com oito escalas, de 1:1.000 a 1:250.000; a série do mapeamento sistemático vai de 1:1.000.000 a 1:25.000, e as escalas de 1:1.000 a 1:10.000 são escalas grandes, fora dessa série, sem norma de nomenclatura de folhas (IBGE, 1999). O tipo de produto distingue, por exemplo, as cartas topográficas matricial e vetorial, a carta ortoimagem e os modelos digitais de terreno e de superfície. O centro de geoinformação identifica um dos cinco CGEOs, cada um com a sua área de suprimento, e o projeto, um dos projetos institucionais. O estado e o município restringem a busca pela interseção espacial entre a folha e o polígono correspondente (Seção 4.2.4), os parâmetros de tempo filtram pela data de publicação e pela data de criação dos metadados, e os demais parâmetros tratam da ordenação e da quantidade de resultados.

Os erros da primeira versão foram observados no conjunto de referência com o Gemma 4 E4B, o modelo escolhido na validação (Seção 4.4), nas 918 execuções das três repetições. Em 154 delas, o modelo deixou de buscar para pedir esclarecimento, embora o *prompt* diga que um único parâmetro basta. Em 72, acrescentou um tipo de produto a pedidos de "cartas"; em 21, alterou o código seguindo o exemplo da descrição ("2901" foi transformado em "2901-2-NE"); em 21, escreveu a escala sem o ponto de milhar do enumerado; e em 18, começou o período na data de execução. A descrição tampouco informava convenções do gabarito, como a de que "última atualização" indica a data de criação. Esses erros motivaram a segunda versão. A ferramenta de busca atende aos requisitos RF1 a RF7 e RF10.

4.2.2 Segunda Versão, Convenções nas Descrições e Recusa

A segunda versão difere da primeira apenas no texto das descrições dos parâmetros e do *prompt* de sistema. As descrições passam a conter as convenções do manual de anotação (Seção 3.3.3.2), que dizem quando preencher cada campo, qual é a forma canônica de cada valor e o que não deduzir. Os códigos de exemplo da primeira versão foram retirados, com uma única exceção ("sf22yd" → "SF-22-Y-D"), e a descrição do estado ganhou a lista de pares de sigla e nome das 27 UFs.

A segunda versão acrescenta a ferramenta `recusar_consulta`, que codifica o limite do domínio. O acervo reúne produtos cartográficos do território brasileiro, e ficam fora dele outros assuntos, outros países e lugares fictícios, os pedidos para produzir ou editar mapas e as armadilhas de vocabulário, como "carta de vinhos", "folha de pagamento" e "escala de plantão". A ferramenta recebe o motivo da recusa, uma frase dirigida ao usuário, e a instrução manda chamar exatamente uma das duas ferramentas, sem pedir esclarecimento. Na saída estruturada da segunda versão, o equivalente é um campo de recusa, sem frase ao usuário, e o texto que enquadra a recusa difere entre as abordagens, porque só o *tool calling* é instruído a buscar "mesmo que de forma vaga"; a comparação entre elas mede, portanto, o mecanismo junto com esse texto. A recusa atende ao requisito RF8.

No conjunto de validação, a falsa recusa do *tool calling* caiu de 32,1% na primeira versão para 0,9% na segunda, e o F1 da recusa subiu de 0,592 para 0,980. A acurácia, porém, quase não mudou (61,5% e 62,8%; p-valor 0,277), porque surgiram erros de forma. A lista de siglas passou a ser usada como formato de saída, e a sigla da UF apareceu no lugar do nome em 41,4% das execuções com estado; a regra do singular foi aplicada como limite igual a 1 em pedidos no plural (8,8%); e a instrução de copiar o código "exatamente como aparece" manteve o prefixo "MI" em 89,4% dos pedidos por MI. Essas causas são hipóteses, porque nenhuma frase foi retirada para testá-las. Os erros são reversíveis por código, sem consulta ao gabarito, e três normalizações elevariam a acurácia a 72,5%, valor

só de diagnóstico, porque nenhuma configuração com elas foi executada.

Os erros da segunda versão eram de forma, e a literatura indica que a correção de um modelo funciona quando o sinal é externo a ele, vindo de uma ferramenta ou de um verificador (HUANG et al., 2024; GOU et al., 2024), como discutido na Seção 2.3.3. Por isso, a versão final transfere parte das convenções para o código, em ferramentas que o modelo consulta e num validador que confere a saída.

4.2.3 Versão Final

A versão final organiza a tradução como um laço de chamadas ao modelo com limites fixos, de seis chamadas ao modelo, três tentativas de busca e duas perguntas por consulta (Figura 5). A cada chamada, o modelo pode consultar as ferramentas auxiliares, cujo resultado volta a ele, pedir esclarecimento ao usuário, recusar a consulta, com a contestação descrita na Seção 4.2.3.8, ou buscar no catálogo, e a busca passa antes pelo validador. Uma resposta só em texto é lida como chamada quando contém uma, escrita com a sintaxe de função ou em JSON; quando não contém, o modelo recebe uma vez o pedido para responder chamando uma ferramenta. Essa leitura faz parte do sistema medido.

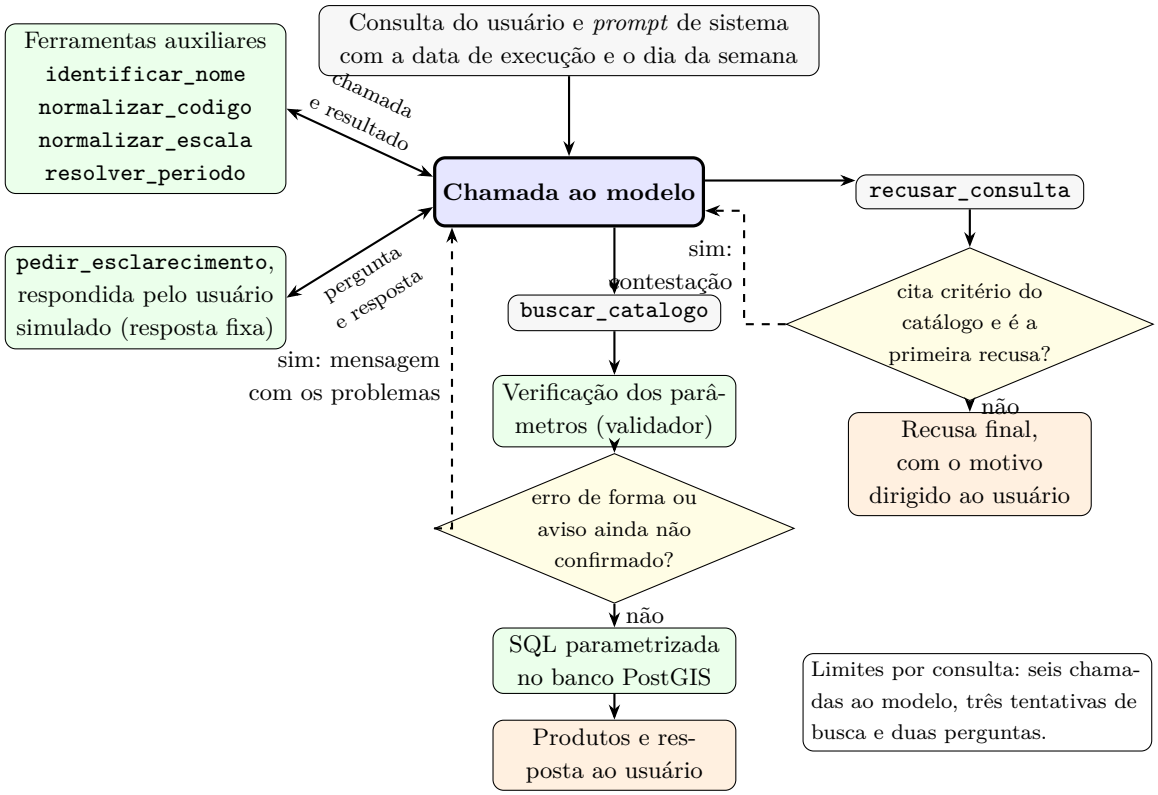


Figura 5 – Laço de chamadas da versão final da tradução

Fonte: Elaborado pelos autores.

As subseções seguintes acompanham a ordem dos passos em que a versão final foi construída e medida.

4.2.3.1 Descrições Revisadas

No primeiro passo da versão final, ainda com uma chamada ao modelo por consulta, as descrições foram revisadas a partir dos erros da Seção 4.2.2. A sigla da UF passou a aparecer só como entrada ("MT" → "Mato Grosso"), o limite no singular passou a exigir ordenação, o código passou a ser escrito sem prefixo nem sufixo, o município, pelo nome completo, e o tipo de produto passou a exigir evidência na consulta. O *prompt* passou a proibir que se deduza um parâmetro de outro, como o estado a partir do município, o CGEO a partir da região ou o projeto a partir do estado, e que se acrescentem ordenação, limite ou tipo não pedidos. Nos resultados, esse passo é a configuração só com as descrições revisadas.

4.2.3.2 Identificação de Nomes

O lugar citado numa consulta é a fonte de ambiguidade mais variada, e nomes de lugares estão entre os argumentos que os modelos mais erram ao chamar ferramentas (LU et al., 2025). A UF pode vir pelo nome ou pela sigla, e algumas siglas coincidem com palavras comuns ("SE", "PA"); "São Paulo" e "Rio de Janeiro" podem designar o estado ou a capital; e há municípios cujo nome contém o de um estado ("Santa Rosa do Piauí"), municípios homônimos, nomes de folha que coincidem com nomes de município e regiões que não são UF ("Nordeste"), casos que a avaliação no acervo real, feita depois do desenvolvimento, também encontrou (Tabela 6). Os conceitos envolvidos são a divisão político-administrativa do IBGE, com 27 UFs e 5.571 municípios, e a nomenclatura das folhas, cujo nome coincide muitas vezes com o de uma localidade, como se observa no acervo com a folha Terra Nova.

A ferramenta `identificar_nome` recebe o trecho da consulta que contém o nome e informa se ele é UF (com o nome por extenso), município (com a UF), folha do acervo (com as escalas em que existe) ou região, e indica em que parâmetro o valor deve entrar. Ela trata formas como "Campinas, SP", separa nomes compostos e, quando não encontra o nome exato, sugere o mais próximo. A ferramenta foi motivada pela sigla escrita no campo de estado na segunda versão e, nos ciclos de desenvolvimento, por municípios pequenos tomados por lugares estrangeiros e recusados; pelo mesmo motivo, o *prompt* passou a dizer que nome de lugar desconhecido não é motivo de recusa e deve ser conferido com a ferramenta. A identificação de nomes atende aos requisitos RF2 e RF3.

4.2.3.3 Normalização de Códigos

A identificação de uma folha pelo código baseia-se na articulação sistemática das folhas (IBGE, 1999). O INOM parte da folha da Carta Internacional do Mundo ao Milionésimo (CIM), identificada pelo hemisfério (N ou S), pela zona de 4° de latitude, com letras de A a U, e pelo fuso de 6° de longitude, numerado de 1 a 60. A partir dela, a folha se subdivide até a escala 1:25.000, com as letras V, X, Y e Z para 1:500.000, as letras de A a D para 1:250.000, os algarismos romanos de I a VI para 1:100.000, os números de 1 a 4 para 1:50.000 e as letras NO, NE, SO e SE para 1:25.000. O MI numera as folhas por escala, de 1 a 46 na escala 1:1.000.000, de 1 a 550 na 1:250.000 e de 1 a 3.036 na 1:100.000, e acrescenta o sufixo de 1 a 4 para a 1:50.000 e NO, NE, SO ou SE para a 1:25.000. A folha SD-23-Y-C-IV-3-NO, por exemplo, corresponde ao MI 2215-3-NO (IBGE, 1999). O nível do código indica a escala, mas a ferramenta não preenche a escala a partir do código, pelo princípio da evidência explícita do manual de anotação, segundo o qual só se preenche o que a consulta diz.

Os códigos geraram erros nas duas versões anteriores. Na primeira, o modelo alterou o código seguindo o exemplo da descrição; na segunda, manteve o prefixo "MI". As consultas também escrevem o INOM de forma compacta ("sf22yd"). A ferramenta `normalizar_codigo` recebe o código como está na consulta, devolve a forma canônica, sem prefixo e, no INOM, em maiúsculas e com hífens, e informa se o código consta no índice do acervo. A normalização de códigos atende ao requisito RF3.

4.2.3.4 Normalização de Escalas

A escala é a razão entre uma distância medida na carta e a distância correspondente no terreno, e escala grande é a de denominador pequeno, que representa uma área menor com mais detalhe. Na primeira versão, o modelo escreveu "1:2000" sem o ponto de milhar do enumerado (Seção 4.2.1), e as consultas trazem formas como "25k" e "25 mil" e expressões qualitativas, como "grande escala" e "maior que 100k", que o manual aceita como um conjunto de escalas, com leituras múltiplas. A ferramenta `normalizar_escala` recebe a expressão e devolve a escala do enumerado, o conjunto de escalas aceito ou a informação de que a escala citada não está no enumerado do catálogo. A normalização de escalas atende ao requisito RF4.

4.2.3.5 Data de Execução e Resolução de Períodos

Um modelo de linguagem não conhece a data em que é executado se ela não lhe for informada. Em todas as versões, o *prompt* de sistema informa a data atual e o dia da semana ("A data atual é . . . "). Na API, essa é a data do dia da execução; nas avaliações, ela foi fixada em cada rodada, em 14/09/2026 nas rodadas dos modelos locais no conjunto de referência e em 24/09/2026 nas dos conjuntos de validação e de teste e nas dos modelos

maiores em nuvem, e o gabarito de cada rodada foi resolvido com a mesma data, para que as rodadas sejam reproduzíveis. Não há ferramenta separada que devolva a data do dia, porque ela já chega ao modelo pelo *prompt*; a dificuldade remanescente estava na aritmética de datas.

A informação da data no *prompt* mostrou-se insuficiente, o que condiz com a dificuldade com datas relativas apontada pelo ToolSandbox (LU et al., 2025). Na primeira versão, o modelo pediu a data em consultas como "ortoimagens de 2 anos atrás" e errou os períodos em três padrões comuns aos modelos. Tratou "esse ano" como começando na data de execução, e não em 1º de janeiro, reduziu "2 anos atrás" a um único dia e trocou o trimestre-calendário por três meses corridos. A acurácia da primeira versão nas consultas de tempo relativo foi de 23,3% no conjunto de referência e de 22,7% no conjunto de validação.

Os conceitos envolvidos são o ano civil, o mês e o trimestre-calendário, as datas de publicação e de criação dos metadados e as regras nomeadas do manual, com as leituras aceitas de cada expressão; pela convenção do manual, "última atualização" indica a data de criação, e "últimos 3 meses" admite noventa dias ou três meses de calendário. A ferramenta `resolver_periodo` recebe a expressão de tempo, devolve o intervalo, com a data inicial e a final, calculado pela tabela do manual a partir da data de execução, e orienta a escolha do campo de data pelo verbo da consulta. A aritmética de datas sai do modelo e passa ao código, e o validador confere o período pela mesma tabela (Seção 4.2.3.7). A resolução de períodos atende ao requisito RF6.

4.2.3.6 Pedido de Esclarecimento

Algumas consultas pedem produtos do acervo sem nenhum critério representável nos parâmetros, como "cartas do Nordeste", em que a região não é estado nem parâmetro do *schema*. A segunda versão proibia perguntar e obrigava a buscar. A ferramenta `pedir_esclarecimento` permite ao modelo fazer uma pergunta ao usuário, opção que o When2Call (ROSS; MAHABALESHWARKAR; SUHARA, 2025) distingue de chamar a ferramenta e de recusar, e o *prompt* a reserva às consultas sem nenhum critério. Nas avaliações e na API, quem responde é um usuário simulado, como no τ-bench (YAO et al., 2025), neste caso com resposta fixa, que diz não ter outros detalhes além do que escreveu e que a busca pode seguir com o que foi dito. Uma pergunta escrita no texto da resposta recebe a mesma resposta, e cada consulta admite no máximo duas perguntas. O pedido de esclarecimento atende ao requisito RF9, com a ressalva de que a pergunta nunca foi respondida por uma pessoa e de que dirigi-la ao usuário real exige conversa em vários turnos (Seção 5.4). A frequência das perguntas no conjunto de teste está na Seção 4.5.2.

4.2.3.7 Verificação dos Parâmetros com Retorno ao Modelo

Como os erros da segunda versão eram de forma e a correção de um modelo funciona melhor com um sinal externo (Seção 4.2.2), os parâmetros passaram a ser conferidos antes da execução da busca, e os problemas encontrados passaram a ser devolvidos ao modelo como resultado da própria chamada de ferramenta, para que ele os perceba e corrija, capacidade avaliada pelo CRITICTOOL (HUANG et al., 2025). A verificação codifica dois princípios do manual (Seção 3.3.3.2), o da evidência explícita e o da forma canônica de cada valor.

O validador separa três classes de problema. Os erros de forma impedem a busca e incluem o valor fora do enumerado, com a sugestão do valor canônico, a sigla ou o nome que não é UF no campo de estado e o parâmetro inexistente. Os avisos de evidência apontam o que foi preenchido sem apoio na consulta, como o estado não citado, o tipo de produto deduzido de "cartas", o CGEO, o projeto, a ordenação ou o limite não pedidos, o período diferente do da tabela do manual e o verbo de tempo contrário ao campo de data. Os avisos de campo ausente apontam o lugar, o código, o tempo, a escala, o tipo, o projeto ou a ordenação que a consulta cita e que não foram preenchidos. Repetir os mesmos parâmetros depois de um aviso confirma a busca, porque o aviso orienta e não obriga, e na terceira tentativa a busca segue com os parâmetros recebidos.

A sigla no campo de estado é tratada como erro de forma, e não como aviso, porque a busca herdada do `prototipo_busca_llm` encontra o estado por trecho do nome, e a sigla corresponde a estados errados; na auditoria dos erros do conjunto de validação, por exemplo, "AM" correspondeu a Amazonas, Amapá e Pernambuco, e o efeito no acervo real está na Seção 4.5.4. Para garantir que o validador não rejeitasse respostas corretas, ele foi aplicado à leitura preferencial do próprio gabarito de desenvolvimento, que não podia receber nenhum erro nem aviso. A primeira implementação do validador apontou problema em 51 das 1.593 consultas do conjunto de validação que admitem busca, e a implementação congelada não apontou problema em nenhuma delas nem nas 298 consultas do conjunto de referência que admitem busca. O controle recebe a mesma verificação, aplicada como nova tentativa depois de uma resposta completa (Seção 3.3.6.4). A verificação dos parâmetros atende aos requisitos RF2 a RF7.

4.2.3.8 Recusa Contestada

Nos ciclos de desenvolvimento, o modelo recusou consultas legítimas que citam um critério do catálogo junto de uma finalidade, como "ortoimagens ... pra uma obra", tomando a finalidade declarada pelo assunto do pedido. Pela regra da recusa contestada, quando a consulta cita tipo de produto, código, escala, CGEO, projeto, UF, município ou nome de folha, a primeira recusa volta ao modelo com a lista desses critérios, e só a segunda é registrada. As palavras "carta" e "mapa" sozinhas não contam como critério,

porque aparecem em muitas consultas fora do domínio. Medida no desenvolvimento, a regra contestaria 12 das 336 consultas fora do domínio do conjunto de validação. A recusa contestada atende ao requisito RF8, na parte que proíbe recusar pedido legítimo.

4.2.3.9 Desenvolvimento e Congelamento

Nos ciclos de desenvolvimento com o Gemma 4 E4B (Seção 3.3.3.3), uma variante com orientações escritas a partir dos erros, no estilo de Zhao et al. (2024), foi retirada por não ter efeito mensurável, com 155 e 155 acertos, com e sem as orientações, nas 160 consultas da amostra estratificada usada nos ciclos, números de dentro da amostra. Em seguida, a versão final foi congelada para o teste (Seção 3.3.5.3).

4.2.4 Predicados Espaciais

A busca aplica interseção espacial entre a geometria da folha e o polígono do estado, do município ou da área de suprimento, com o predicado de interseção do PostGIS (POSTGIS DEVELOPMENT TEAM, 2001). Na avaliação pela execução no acervo real, esse filtro foi executado com as malhas de estados e municípios do IBGE em 1.071 buscas medidas (Seção 4.5.4), com quatro particularidades. A geometria da folha é a caixa envolvente, o retângulo de coordenadas informado pelo CSW, e 86 registros sem caixa válida ficam fora dos filtros espaciais. A interseção inclui as folhas que apenas tocam o limite do estado ou do município, e por isso folhas vizinhas aparecem no resultado. A área de suprimento aproximada (Seção 3.3.2) corresponde à região onde cada CGEO tem produtos no catálogo, e não à área de suprimento real. O estado e o município, por fim, são encontrados por trecho do nome, o que faz "Santa Maria" corresponder a vários municípios (Tabela 6).

Os predicados relacionais, como "perto de", "num raio de" e "ao norte de", exigem geocodificar o lugar citado, acrescentar parâmetros ao *schema* e implementar operadores de distância e de direção, e ficam como trabalho futuro (Seção 5.4). A interseção espacial atende ao requisito RF2.

4.2.5 Síntese das Funcionalidades

A Tabela 7 relaciona cada funcionalidade ao erro ou à ambiguidade que resolve, ao conceito do domínio que codifica, ao requisito que atende e à versão em que entrou na busca.

Tabela 7 – Funcionalidades da busca, erro ou ambiguidade que resolvem, conceito, requisito e versão

Funcionalidade	**Erro ou ambiguidade que resolve**	**Conceito do domínio**	**Requisito**	**Versão**
Busca no catálogo	modelo escrevendo SQL; dicionário fixo de normalização	enumerado do catálogo, tipos da ET-PCDG, interseção espacial	RF1 a RF7, RF10	primeira
Convenções nas descrições	convenções do gabarito não informadas ao modelo	regras do manual de anotação	RF2 a RF7	segunda; revisadas na final
Recusa	não buscar sem explicar; saída estruturada sempre busca	limites do acervo	RF8	segunda
Identificação de nomes	sigla, homônimos, estado ou capital, região	divisão político-administrativa; nomes das folhas	RF2, RF3	final
Normalização de códigos	código copiado, prefixo, INOM compacto	articulação INOM e MI	RF3	final
Normalização de escalas	grafias e expressões qualitativas	escala como razão; enumerado do catálogo	RF4	final
Resolução de períodos	datas relativas erradas	tempo civil; publicação e criação	RF6	final
Pedido de esclarecimento	consulta sem critério	região não é UF	RF9	final
Verificação dos parâmetros	erros de forma e campos sem evidência	evidência explícita; forma canônica	RF2 a RF7	final
Recusa contestada	recusa de pedido legítimo	critérios do catálogo	RF8	final

Fonte: Elaborada pelos autores.

4.3 Instalação

Os quatro modelos locais e o Phi-4 14B da solução do Maj Diniz foram instalados pelo Ollama com as etiquetas oficiais, listadas na Tabela 8. Os critérios de escolha e a configuração estão na Seção 3.3.4, e as características de cada modelo, na Tabela 2. Foram usadas três versões do Ollama. Na GPU T4 do Google Colab, as rodadas do Gemma 4 E4B e a comparação no conjunto de referência usaram a 0.34.4, e as rodadas do Gemma 4 E2B e do Qwen 3 4B no conjunto de teste, a 0.35.0; a estação local usou a 0.34.0. Na T4, todos os modelos couberam inteiros na GPU; na estação local, nenhum dos três modelos executados foi carregado inteiro nela. A ocupação de memória do Gemma 4 diferiu entre os dois ambientes com a mesma etiqueta, por causa não registrada, e a VRAM necessária a cada modelo não foi tratada como requisito medido.

Tabela 8 – Modelos instalados, memória ocupada e emissão das chamadas de ferramenta

Modelo	**Etiqueta no Ollama**	**Memória na T4**	**Estação local (carregado; parcela na GPU)**	**Emissão da chamada de ferramenta**
Qwen 3 4B Instruct 2507	`qwen3:4b-instruct-2507-q4_K_M`	3,18 GB	3,53 GB; 66%	pelo canal de ferramentas; na versão final, às vezes escrita no texto
Gemma 4 E4B	`gemma4:e4b-it-qat`	3,09 GB	6,04 GB; cerca de um quarto	pelo canal de ferramentas; na versão final, escrita no texto com mais frequência
Gemma 4 E2B	`gemma4:e2b-it-qat`	1,65 GB	3,93 GB; 38%	pelo canal de ferramentas; na versão final, às vezes escrita no texto
Mistral Nemo 12B	`mistral-nemo:12b`	7,48 GB	não executado	deixou de chamar a ferramenta em muitas execuções, várias vezes descrevendo em texto a chamada que faria
Phi-4 14B (solução do Maj Diniz)	`phi4:14b`	9,73 GB	não executado	sem *tool calling* nativo no Ollama; usado com saída estruturada

Fonte: Elaborada pelos autores a partir dos registros de cada rodada (CORRÊA; SANTANA, 2026).

Nota: Memória do modelo carregado com contexto de 4.096 *tokens*, informada pelo Ollama. Na T4, a memória registrada para os dois Gemma 4 é menor que o tamanho do arquivo na Tabela 2. A causa dessa diferença não foi registrada, e o valor não entra em nenhuma conclusão.

Os modelos diferiram na emissão das chamadas pelo *tool calling* nativo (Seção 2.3.1). O Qwen 3 4B e os dois Gemma 4 emitiram as chamadas pelo canal de ferramentas. O Mistral Nemo 12B descreveu em texto a chamada que faria, sem emiti-la, em 357 das 918 execuções do conjunto de referência; para "preciso da carta MI 2965-2-NE do segundo cgeo", respondeu que chamaria a ferramenta com o código e o segundo CGEO. O Phi-4 14B não tem *tool calling* nativo no Ollama e, na solução do Maj Diniz, é usado com saída estruturada. No conjunto de validação, com a segunda versão, só 13 das 1.929 respostas do Gemma 4 E4B trouxeram a chamada escrita no texto. A versão final lê essas chamadas (Seção 4.2.3), e a frequência delas no conjunto de teste está na Seção 4.5.2. Nenhum modelo produziu, na primeira versão, argumento com campo inexistente ou tipo inválido.

4.4 Validação dos Modelos

Esta seção trata dos conjuntos de consultas construídos para validar os modelos e da validação dos quatro modelos locais com a primeira versão no conjunto de referência. O conjunto de validação, um desses conjuntos, não se confunde com o validador da versão final (Seção 4.2.3.7).

A Tabela 9 resume os três conjuntos, cuja construção está na Seção 3.3.5. Das 310 consultas do conjunto de referência, quatro são observacionais e ficam fora da pontuação, e por isso 306 são pontuadas.

Tabela 9 – Conjuntos de consultas construídos para a validação e a avaliação

Conjunto	**Consultas**	**Construção**	**Fora do domínio**	**Sem critério**	**Leituras múltiplas**	**Papel**
Referência	310 (306 pontuadas)	três origens disjuntas (Seção 3.3.5.1); gabarito dos autores pelo manual, auditado	8	—	67	escolha do modelo; comparação com a solução do Maj Diniz; desenvolvimento
Validação	1.929	1.950 alvos sorteados do BDGEx e do IBGE, com o gabarito fixado antes da redação; redação e anotação às cegas por agentes; filtro de compatibilidade	336	146	471	teste da segunda versão; desenvolvimento da versão final
Teste	945	mesma receita, 975 alvos com outras sementes e outras sessões de agentes	163	65	233	medida única da versão final e da extensão a outros modelos

Fonte: Elaborada pelos autores. Listagens, manual e auditoria em Corrêa e Santana (2026).

A qualidade do gabarito foi verificada pelo procedimento da Seção 3.3.5.1. No conjunto de referência, a anotação independente às cegas concordou com o gabarito em 99,4% dos valores de campo e encontrou leitura compatível em 300 das 310 consultas. Na decisão de buscar ou não, concordou em 302 consultas, com kappa de 0,658, que passou a 1,000 depois da adjudicação de 15 consultas; o kappa desconta a concordância esperada ao acaso, que é alta quando quase todas as consultas pedem busca. A principal divergência foi de classificação; o gabarito tratava como observacionais as oito consultas que a adjudicação passou a classificar como fora do domínio. Além disso, um dos autores conferiu à mão uma amostra de 40 consultas e confirmou o gabarito de todas. Nos conjuntos de validação e de teste, antes do filtro de compatibilidade, o kappa na decisão de buscar foi de 1,000

e 1,000, e a segunda anotação, feita fora do filtro, foi compatível com o gabarito em 100,0% das 383 e em 100,0% das 191 consultas da amostra. Esses números são altos por construção, porque a mesma família de modelos redigiu e anotou as consultas, e nenhuma pessoa conferiu esses dois conjuntos. Na auditoria dos erros, que julgou pelo manual 160 execuções erradas do conjunto de validação, sorteadas por configuração e estratificadas por família de consulta, 155 (96,9%; IC 95% de 92,9% a 98,7%) eram erros reais do modelo, 5 tinham gabarito discutível e nenhuma tinha gabarito contrário ao manual.

A Tabela 10 traz o resultado da primeira versão nos quatro modelos locais, executada em três repetições na T4. Três modelos não apresentaram diferença significativa entre si (p-valor 0,101 entre o Qwen 3 4B e o Gemma 4 E4B, 0,305 entre os dois Gemma 4 e 0,569 entre o Qwen 3 4B e o Gemma 4 E2B). O Mistral Nemo 12B, o maior dos quatro, obteve acurácia menor, com diferença significativa para cada um dos outros três. Parte relevante dessa diferença decorre da chamada descrita em texto (Seção 4.3), mas a sua acurácia foi menor também com saída estruturada. Entre os modelos locais, o desempenho não acompanhou o porte. O Mistral Nemo 12B é também o mais antigo dos quatro, lançado em julho de 2024 (Tabela 2), mas com quatro modelos não se separa o efeito da geração do efeito do porte, e a coincidência fica como observação.

Tabela 10 – Primeira versão nos quatro modelos locais, saída estruturada e solução do Maj Diniz no conjunto de referência

Configuração	**Acurácia (IC 95%)**	**F1 dos campos**	**Acerto fora do domínio**	**Latência mediana na T4**	**Acurácia com saída estruturada**	***p*-valor (*tool calling* e saída estruturada)**
Primeira versão (tool calling)						
Qwen 3 4B	54,2% (48,6% a 59,7%)	0,666	100,0%	0,8 s	60,1%	0,038
Gemma 4 E4B	60,2% (54,7% a 65,6%)	0,793	100,0%	0,7 s	75,2%	inferior a 0,001
Gemma 4 E2B	56,5% (50,9% a 62,0%)	0,765	62,5%	0,4 s	53,9%	0,488
Mistral Nemo 12B	26,8% (22,1% a 32,0%)	0,451	100,0%	2,4 s	52,0%	inferior a 0,001
Solução do Maj Diniz (saída estruturada)						
Phi-4 14B	58,8% (53,2% a 64,2%)	0,856	0,0%	12,6 s	—	—

Fonte: Elaborada pelos autores.

Nota: Primeira versão com três repetições (média); saída estruturada e solução do Maj Diniz com uma. O p-valor é o do teste de McNemar entre o *tool calling* e a saída estruturada no mesmo modelo, com uma observação por consulta. Acerto fora do domínio nas oito consultas fora do domínio.

O Qwen 3 4B teve a menor precisão nos campos, porque acrescentou ordenação e limite não pedidos, e declarou fora do escopo consultas legítimas em 27 execuções. As três repetições deram a mesma resposta em 287 a 306 das 306 consultas. Pelo desempate do F1 (Seção 3.3.6.2), o Gemma 4 E4B foi escolhido para o desenvolvimento da segunda versão e da versão final. Na estação local, nas 59 consultas pontuadas do subconjunto manual (as observacionais ficam fora), a pontuação coincidiu com a da primeira repetição na T4 em 57 delas em cada um dos três modelos executados, e a qualidade praticamente não mudou com o *hardware*.

Com a primeira versão e as mesmas consultas, os modelos maiores em nuvem acertaram mais. O Qwen 3.8 27B (QWEN TEAM, 2026) acertou 95,1%, e o GPT-OSS 120B e o GPT-OSS 20B (OPENAI, 2025), 89,9% e 89,2%. Esses resultados indicam que a tarefa, com o gabarito adotado, é solúvel por modelos maiores, mas esses modelos não atendem ao RNF1, que exige execução local, e ficam fora da recomendação. As colunas da Tabela 10 sobre a saída estruturada e a solução do Maj Diniz são discutidas na Seção 4.5.1.

4.5 Avaliação de Performance

Esta seção avalia a busca em comparação com a solução do Maj Diniz e isoladamente, pelos métodos da Seção 3.3.6, com as versões no conjunto de teste, o teste A/B, a execução no acervo real e o tempo de processamento.

4.5.1 Comparação com a Solução do Maj Diniz

A solução do Maj Diniz acertou 58,8% das consultas do conjunto de referência (Tabela 10), sem diferença significativa para a primeira versão com os três melhores modelos locais (p-valor 0,805, 0,614 e 0,206 contra o Gemma 4 E4B, o Gemma 4 E2B e o Qwen 3 4B). O empate não demonstra equivalência (Seção 3.3.6.2).

A primeira versão chegou a essa acurácia com modelos de 2,3 a 4,5 bilhões de parâmetros efetivos, mais de quinze vezes mais rápidos, com mediana de 0,4 s a 0,8 s por consulta na T4, contra 12,6 s da solução do Maj Diniz, sem dicionário nem exemplos e com recusa explícita. A redução de porte e de tempo decorreu da troca de modelo, e não do *tool calling*. Na solução do Maj Diniz, a primeira resposta foi rejeitada pela conferência do *schema* e exigiu nova tentativa em 295 das 306 consultas, o dicionário alterou o texto de 68 consultas antes de chegarem ao modelo, e o *fallback* por expressões regulares foi acionado em 10 consultas.

Com uma chamada ao modelo, a saída estruturada acertou mais que o *tool calling* em três dos quatro modelos, com diferença significativa, e no Gemma 4 E2B as duas empataram (Tabela 10). A vantagem decorreu em boa parte da liberdade que o *tool calling* tem de não buscar. Das 62 consultas que só a saída estruturada acertou com o Gemma 4

E4B, em 32 o *tool calling* não buscou, quase sempre para pedir esclarecimento. Só o *tool calling* recusou consultas fora do domínio. A versão final só foi comparada à solução do Maj Diniz no conjunto de referência, dentro da amostra, com acurácia de 97,7%, porque essa solução, reproduzida com as três diferenças deliberadas da Seção 3.3.6.3, não foi executada nos conjuntos de validação e de teste.

4.5.2 As Versões no Conjunto de Teste

A Tabela 11 reúne os passos da versão final, o controle e a extensão aos outros dois modelos no conjunto de teste. Com o Gemma 4 E4B, a diferença da primeira para a segunda versão não foi significativa (p-valor 0,058), como no conjunto de validação, empate interpretado conforme a regra de decisão (Seção 3.3.6.2). Os três passos seguintes acrescentaram acurácia com diferença significativa. Em relação ao passo anterior, a diferença foi de +13,1 p.p. só com as descrições revisadas, o maior ganho isolado, obtido apenas com o texto das descrições, de +7,8 p.p. com as ferramentas auxiliares, passo que também introduz o usuário simulado e o laço de chamadas, e de +11,3 p.p. com a verificação dos parâmetros e a recusa contestada. Com a correção de Holm nas nove comparações do plano (Seção 3.3.6.2), as diferenças significativas continuam significativas.

Tabela 11 – Passos da versão final, controle e outros modelos no conjunto de teste

Configuração	**Acurácia (IC 95%)**	***p*-valor do passo**	**Acurácia no domínio**	**F1 da recusa**	**Falsa recusa**	**Latência mediana na T4 (s)**	**Chamadas ao modelo**
Gemma 4 E4B							
Primeira versão	58,6% (55,5% a 61,7%)	—	45,9%	0,562	35,4%	0,85	1,00
Segunda versão	62,1% (59,0% a 65,2%)	0,058	53,3%	0,991	0,4%	0,91	1,00
Só com as descrições revisadas	75,2% (72,4% a 77,9%)	$<0{,}001$	70,3%	0,985	0,7%	0,90	1,01
Com as ferramentas auxiliares	83,1% (80,5% a 85,3%)	$<0{,}001$	79,8%	0,943	1,8%	1,37	1,82
Versão final (com a verificação dos parâmetros)	94,4% (92,7% a 95,7%)	$<0{,}001$	94,4%	0,969	0,3%	1,47	2,06
Controle	87,8% (85,6% a 89,8%)	$<0{,}001$	84,4%	0,901	5,0%	0,78	1,23
Gemma 4 E2B							
Versão final	80,4% (77,8% a 82,8%)	—	79,4%	0,869	4,3%	1,00	2,37
Controle	65,4% (62,3% a 68,4%)	$<0{,}001$	55,4%	0,767	13,4%	0,44	1,62
Qwen 3 4B							
Versão final	73,5% (70,6% a 76,3%)	—	67,2%	0,864	6,4%	3,15	2,86
Controle	88,3% (86,0% a 90,2%)	$<0{,}001$	84,8%	0,863	6,8%	0,65	1,33

Fonte: Elaborada pelos autores. Conjunto de teste, uma execução por consulta, GPU T4.

Nota: O *p*-valor do passo compara cada linha com a anterior no mesmo modelo (teste de McNemar); nas linhas do controle, com a versão final do mesmo modelo. O controle é a saída estruturada com as mesmas descrições e a mesma verificação. As latências do Gemma 4 E2B e do Qwen 3 4B vêm de outra versão do Ollama e não se comparam às do Gemma 4 E4B.

Na comparação principal, a versão final acertou 94,4% das consultas (IC 95% de 92,7% a 95,7%), e o controle, 87,8%, com 95 consultas acertadas só pela versão final e 33 só pelo controle (p-valor $<0{,}001$). A vantagem decorreu inteiramente das consultas do domínio, em que o controle declarou fora do escopo 5,0% das consultas. O controle recusou todas as consultas fora do domínio, contra 95,1% da versão final, e acertou 95,4% das consultas sem critério, contra 92,3%, quase sempre declarando-as fora do escopo, o que a pontuação aceita. A diferença mede o mecanismo junto com as peças que só o *tool*

calling recebe (Seção 3.3.6.4).

A decomposição entre a primeira decisão e a resposta final indica quanto do acerto se deve ao modelo e quanto ao código. A primeira decisão da versão final acertou 83,0%, contra 74,6% do controle, numa comparação exploratória, e já inclui o retorno das ferramentas auxiliares, de modo que não mede o modelo sem o código. O retorno do validador produziu diferença de +11,4 p.p. na versão final e de +13,2 p.p. no controle. Das consultas corrigidas pelo retorno na versão final, 53,6% receberam do validador o valor a usar, e nelas o acerto se deve ao código, que implementa as convenções do manual.

No laço, o Gemma 4 E4B escreveu ao menos uma chamada no texto da resposta em 236 consultas (25,0%), lidas como chamadas; essa leitura faz parte do sistema medido, e não se sabe quanto a versão final acertaria sem ela, porque o diálogo seria outro. O modelo perguntou ao usuário simulado em 133 consultas, mais que as 65 consultas sem critério, as únicas a que o *prompt* reserva a pergunta. A recusa contestada foi acionada em 20 consultas, e em 3 consultas fora do domínio a recusa estava certa, foi contestada e o modelo passou a buscar, o que explica parte da queda do *recall* da recusa em relação à segunda versão (100,0%). Numa leitura exploratória, a acurácia nas consultas de tempo relativo foi de 22,3% na primeira versão e de 91,4% na versão final.

Com o Gemma 4 E2B e o Qwen 3 4B, a mesma versão final congelada foi executada nas condições da Seção 3.3.6.4. Em relação à primeira versão, a diferença de acurácia do *tool calling* foi de +22,3 p.p. com o Gemma 4 E2B e de +16,9 p.p. com o Qwen 3 4B, mas a comparação principal dependeu do modelo. Com o Gemma 4 E2B, a versão final superou o controle (211 consultas acertadas só por ela, contra 69; p-valor $< 0{,}001$). Com o Qwen 3 4B, a diferença foi de +14,7 p.p. a favor do controle (p-valor $< 0{,}001$), que já estava à frente na primeira decisão (68,3% contra 61,4%); segundo uma análise exploratória dos erros, o modelo acrescentou ordenações não pedidas com o *tool calling* e, avisado pelo validador, repetiu os mesmos parâmetros, o que pela regra confirma a busca.

Numa comparação exploratória, a acurácia nas consultas do domínio caiu do conjunto de referência, dentro da amostra, para o conjunto de teste, de 98,3% para 94,4% na versão final e de 94,0% para 84,4% no controle, uma queda esperada em regras ajustadas sobre os erros do desenvolvimento e maior no controle. Os erros restantes da versão final, lidos depois do teste e sem alterá-la, concentram-se no parâmetro de palavra-chave preenchido com o que não é nome de carta, como uma região, um projeto ou uma escala, que o validador não confere, nas consultas fora do domínio em que o modelo buscou e nas expressões de tempo que a tabela do manual não cobre. Corrigi-los a partir do conjunto de teste o transformaria em conjunto de desenvolvimento, e a medição do efeito dessas correções fica para um conjunto novo (Seção 5.4).

4.5.3 Teste A/B

A Tabela 12 resume o teste A/B. Nos 24 pares de rodadas em que o mesmo modelo executou as duas abordagens, a saída estruturada foi melhor em 8, o *tool calling* em 8, e houve 8 empates. Nos 19 pares medidos fora da amostra, isto é, num conjunto que não serviu ao desenvolvimento da versão avaliada, as contagens foram de 6, 5 e 8.

Tabela 12 – Teste A/B entre o *tool calling* e a saída estruturada

Placar dos pares (teste de McNemar)				
Pares	**Saída estruturada melhor**	***Tool calling* melhor**	**Empate**	
Todos (24)	8	8	8	
Fora da amostra (19)	6	5	8	
Pares com diferença significativa que não aparecem nas Tabelas 10 e 11				
Conjunto, versão e modelo	**Acurácia da saída estruturada**	**Diferença (p.p.)**	***p*-valor**	**F1 da recusa (*tool calling*; saída estruturada)**
Referência, primeira versão, GPT-OSS 20B (nuvem)	77,5%	-11,8	$< 0{,}001$	0,889; 0,000
Validação, segunda versão, Gemma 4 E4B	72,4%	+9,6	$< 0{,}001$	0,980; 0,723
Teste, primeira versão, Gemma 4 E2B	36,5%	-21,6	$< 0{,}001$	0,795; 0,000
Teste, primeira versão, Qwen 3 4B	51,4%	-5,2	0,014	0,903; 0,000
Teste, segunda versão, Gemma 4 E4B	73,8%	+11,6	$< 0{,}001$	0,991; 0,738

Fonte: Elaborada pelos autores.

Nota: Diferença = saída estruturada menos *tool calling*; valor negativo é vantagem do *tool calling*. Os outros seis pares com diferença significativa fora da amostra são os do Gemma 4 E4B, do Qwen 3 4B e do Mistral Nemo 12B na primeira versão (Tabela 10) e os da versão final com os três modelos (Tabela 11).

A abordagem que acerta mais depende da versão e do modelo. Com a primeira versão, no conjunto de referência, a saída estruturada acertou mais em três dos quatro modelos locais, e as duas empataram no Gemma 4 E2B (Seção 4.5.1). No conjunto de teste, com o Gemma 4 E2B e o Qwen 3 4B, a vantagem do *tool calling* na primeira versão decorreu sobretudo das consultas fora do domínio, que a saída estruturada da primeira versão não tem como recusar; por isso o F1 da recusa da saída estruturada é nulo nas linhas da primeira versão. No conjunto de validação, com a primeira versão e o Gemma 4 E4B, as duas abordagens empataram por compensação, porque a saída estruturada acertava

mais nas consultas do domínio e só o *tool calling* recusava as consultas fora dele. Como a acurácia geral pondera as duas partes pela proporção de consultas fora do domínio, as acurácias das duas abordagens igualam-se quando essa proporção vale 18,0%, e esse conjunto tinha 17,4%. Entre os modelos maiores em nuvem, o GPT-OSS 20B acertou mais com *tool calling* também sem as consultas fora do domínio. Com a segunda versão, a saída estruturada obteve acurácia significativamente maior, numa comparação em que o texto de recusa difere entre as abordagens (Seção 4.2.2). Com a versão final, o *tool calling* obteve acurácia maior com os dois Gemma 4 e menor com o Qwen 3 4B.

O ganho que se repetiu foi o da recusa. Quando as duas abordagens podiam recusar, o *tool calling* teve o maior F1 da recusa em todos os pares fora da amostra, e com o Qwen 3 4B na versão final a diferença entre os dois valores foi mínima. Três diferenças a favor do *tool calling* ficam fora da acurácia ou só entram nela junto com outras peças, a saber, a frase de recusa dirigida ao usuário, o canal próprio para perguntar, exercitado só com o usuário simulado, e a verificação recebida no meio da resposta, como resultado da própria chamada, que entra na acurácia da versão final junto com o código das ferramentas.

4.5.4 Avaliação pela Execução no Acervo Real

A avaliação pela execução no acervo real é uma análise complementar, sem novas rodadas de modelos (Seção 3.3.6.6). As buscas do gabarito e as emitidas por cada configuração no conjunto de teste foram executadas no acervo, sem alteração da consulta SQL, e a Tabela 13 compara o acerto por resultado com o acerto por parâmetros.

Tabela 13 – Acerto por resultado e por parâmetros no acervo real do BDGEx

Configuração	**Acerto por resultado (todas as consultas)**	**Acerto por parâmetros (gabarito com cartas)**	**Acerto por resultado (gabarito com cartas)**
Gemma 4 E4B			
Primeira versão	64,8%	47,2%	50,6%
Segunda versão	77,8%	54,7%	62,9%
Só com as descrições revisadas	87,3%	71,4%	80,0%
Com as ferramentas auxiliares	90,8%	83,9%	88,6%
Versão final	97,0%	94,3%	96,7%
Controle	91,3%	87,0%	87,7%
Gemma 4 E2B			
Versão final	87,5%	83,5%	85,0%
Controle	72,9%	63,7%	66,4%
Qwen 3 4B			
Versão final	85,8%	76,5%	83,3%
Controle	90,8%	88,8%	89,4%

Fonte: Elaborada pelos autores. Conjunto de teste (945 consultas; 545 com gabarito com cartas) executado no acervo real.

Em 237 consultas, todas as leituras aceitas do gabarito devolvem zero cartas, por causa do projeto ausente do CSW, de combinações de filtros sem carta em comum e de tipos e escalas ausentes do acervo. Nelas, qualquer busca que também devolva zero cartas conta como resultado igual, e por isso a comparação justa é a restrita às 545 consultas cujo gabarito tem cartas.

A pontuação por parâmetros não superestimou o acerto. Na versão final, 717 das 718 buscas com parâmetros certos devolveram as mesmas cartas do gabarito. Entre as buscas com parâmetros errados, 26 devolveram as mesmas cartas, a maioria por limitação do acervo, e não por equivalência entre os parâmetros. Delas, 17 devolveram zero cartas nos dois lados, 4 trocaram o campo de data, o que só coincide porque o CSW grava uma única data por registro, 2 mudaram só o limite ou a direção da ordenação, e só 3 eram realmente inofensivas. Como quase nenhum erro de parâmetro é inofensivo, a métrica por parâmetros não é tratada como conservadora.

Pelo acerto por resultado, nas consultas cujo gabarito tem cartas, a versão final continua à frente do controle com os dois Gemma 4, e o controle continua à frente com o Qwen 3 4B; com o Gemma 4 E4B, a versão final devolveu as mesmas cartas do gabarito em 96,7% dessas consultas. Nem toda a ordem se mantém, porque o passo com as ferramentas auxiliares e o controle trocam de posição em relação à Tabela 11.

A avaliação mostra também por que o validador bloqueia a sigla no campo de estado (Seção 4.2.3.7). Numa simulação com as 27 siglas, só 5 devolveram o mesmo re-

sultado que o nome da UF, 10 corresponderam ao próprio estado e também a outros, 10 não corresponderam a estado algum e 2 corresponderam a um estado errado, como “SP”, que corresponde a Espírito Santo. Nas rodadas, 18 das 51 execuções com sigla mudaram o resultado, todas na segunda versão. Em 362 das 782 buscas do gabarito, por fim, o resultado passa da primeira página, e o usuário vê só as cartas mais recentes. Pelas limitações do CSW descritas na Seção 3.3.6.6, a avaliação mede se a busca devolve as mesmas cartas que a do gabarito, e não se o usuário encontrou o que queria.

4.5.5 Tempo de Processamento

Na T4, as medianas de latência e as chamadas ao modelo por consulta estão na Tabela 11, e a versão final, com cerca de duas chamadas ao modelo por consulta, demora mais que o controle. Com uma chamada ao modelo, as duas abordagens têm latências da mesma ordem de grandeza, em geral com décimos de segundo a favor da saída estruturada, exceto no Mistral Nemo 12B (2,39 s com *tool calling* contra 1,00 s), que respondeu com textos longos descrevendo a chamada.

Na estação local, nas consultas pontuadas do subconjunto manual, dentro da amostra e com a GPU por vezes compartilhada com outros programas, a versão final teve mediana de 5,63 s com o Gemma 4 E4B, contra 3,38 s do controle, de 1,75 s com o Gemma 4 E2B, contra 0,80 s, e de 11,33 s com o Qwen 3 4B, contra 3,21 s. O tempo de execução da consulta SQL no acervo real teve mediana de 49 ms (percentil 95 de 127 ms), enquanto a resposta do modelo demora alguns segundos. Só se comparam latências com a mesma GPU, a mesma versão do Ollama e a mesma parcela do modelo na GPU (Seção 3.3.6.5).

4.6 Discussão dos Resultados

A discussão retoma os resultados pelos requisitos, pelos ganhos observados, pelas questões do trabalho, pela recomendação por cenário e pela validade do que foi medido.

4.6.1 Atendimento dos Requisitos

A Tabela 14 mostra como cada requisito do Quadro 1 foi atendido. Todos foram atendidos, a maior parte com ressalvas que vêm dos limites do acervo coletado pelo CSW, do usuário simulado no lugar de pessoas ou da dependência do modelo, discutidas na Seção 4.6.5.

Tabela 14 – Atendimento dos requisitos

Req.	**Como foi atendido**	**Evidência**	**Situação**
RF1	API com tradução e busca; execução no acervo real	Seções 3.2.1 e 4.5.4	atendido
RF2	identificação de nomes, verificação da sigla e interseção espacial	Seções 4.2.3.2, 4.2.3.7 e 4.2.4	atendido com ressalva (homônimos sem desambiguação; sem predicados relacionais)
RF3	normalização de códigos e índice do acervo	Seção 4.2.3.3	atendido com ressalva (MI com e sem zero à esquerda)
RF4	normalização de escalas	Seção 4.2.3.4	atendido
RF5	descrições revisadas e verificação	Seções 4.2.3.1 e 4.2.3.7	atendido com ressalva (projeto ausente do CSW)
RF6	data de execução no *prompt* e resolução de períodos	Seção 4.2.3.5	atendido com ressalva (expressões fora da tabela; uma data por rodada)
RF7	descrições e avisos de ordenação e limite não pedidos	Seções 4.2.3.1 e 4.5.2	atendido com ressalva (depende do modelo)
RF8	recusa com motivo e recusa contestada	Seções 4.2.2, 4.2.3.8 e 4.5.2	atendido com ressalva (*recall* da recusa menor na versão final)
RF9	pedido de esclarecimento	Seção 4.2.3.6	atendido com ressalva (só com usuário simulado)
RF10	lista vazia com os critérios usados	Seção 3.2.1	atendido, não pontuado
RF11	registro por consulta	Seção 3.2.1	atendido
RNF1	modelos locais pelo Ollama	Seção 4.3	atendido
RNF2	latência por ambiente	Seção 4.5.5	atendido com ressalva (sem limiar fixado)
RNF3	esquema do `prototipo_busca_llm` e metadados do BDGEx	Seções 3.2.1 e 4.5.4	atendido com ressalva (conformidade não verificada à parte)
RNF4	registros, congelamento e repontuação	Seção 3.3.5.3	atendido
RNF5	testes automatizados e integração contínua	Seção 3.2.1	atendido
RNF6	modelo e abordagem por configuração	Seção 3.2.1	atendido

Fonte: Elaborada pelos autores.

4.6.2 Ganhos Observados

No conjunto de teste, com o Gemma 4 E4B, o maior ganho isolado da versão final decorreu das descrições revisadas, isto é, das convenções do domínio escritas em texto, sem nenhuma ferramenta nova (Tabela 11). Os passos seguintes, com as ferramentas auxiliares

e com a verificação dos parâmetros, acrescentaram ganhos menores, também significativos depois da correção de Holm. Nesses dois passos, parte do que o modelo errava de forma sistemática, como a aritmética de datas, a sigla da UF no campo de estado e a forma canônica dos códigos, foi transferida ao código. Esses ganhos condizem com a dificuldade dos modelos com datas relativas e nomes de lugares apontada pelo ToolSandbox (LU et al., 2025) e com a eficácia da correção guiada por um sinal externo ao modelo (GOU et al., 2024; HUANG et al., 2024), discutidas na Seção 2.3.3. Pelo acerto por resultado no acervo real, a posição relativa da versão final e do controle manteve-se nos três modelos (Seção 4.5.4).

Em relação à solução do Maj Diniz, a primeira versão não apresentou diferença significativa de acurácia, e a redução de porte e de tempo decorreu da troca de modelo (Seção 4.5.1). O único ganho do mecanismo de *tool calling* que se repetiu em todos os pares fora da amostra foi o da recusa das consultas fora do domínio (Seção 4.5.3), que corresponde à decisão de não chamar a ferramenta de busca, avaliada como abstenção pelo BFCL (PATIL et al., 2025) e distinguida pelo When2Call (ROSS; MAHABALESHWARKAR; SUHARA, 2025) das decisões de chamar e de perguntar. O restante da vantagem da versão final sobre o controle não se separa das peças do domínio que só ela recebe (Seção 3.3.6.4).

4.6.3 Respostas às Questões do Trabalho

As respostas seguem a ordem das questões da Seção 1.2.3:

1. *O tool calling melhora a tradução em relação à solução do Maj Diniz?* Parcialmente. Com a primeira versão, o *tool calling* não diferiu significativamente em acurácia da solução do Maj Diniz no conjunto de referência, com modelos menores e mais rápidos e com recusa explícita, que aquela solução não tem; a redução de tempo, porém, decorre da troca de modelo. A versão final só foi comparada a ela no conjunto de referência, dentro da amostra (Seção 4.5.1).

2. *No teste A/B, o tool calling obtém acurácia maior que a da saída estruturada?* Parcialmente. A abordagem que acerta mais depende da versão e do modelo, e o ganho que se repetiu fora da amostra foi o da recusa (Seção 4.5.3).

3. *No teste A/B, o tool calling reduz o tempo de processamento em relação à saída estruturada?* Não. Com uma chamada ao modelo, não houve ganho de tempo, e a versão final faz cerca de duas chamadas ao modelo por consulta e demora mais (Seção 4.5.5).

4. *As ferramentas auxiliares do domínio aumentam a acurácia dos modelos locais?* Parcialmente. Com o Gemma 4 E4B, o passo com as ferramentas auxiliares acres-

centou acurácia com diferença significativa, mas esse passo inclui também o pedido de esclarecimento, o usuário simulado e o laço de chamadas, sem isolá-los, e não foi medido com outros modelos. Com o Qwen 3 4B, o controle, sem as ferramentas auxiliares, acertou mais que a versão final. O efeito depende, portanto, do modelo (Seção 4.5.2), com as ressalvas da Seção 3.3.6.4.

Na análise complementar no acervo real, que não corresponde a uma questão planejada, verificou-se se as buscas produzidas devolvem as mesmas cartas que as buscas do gabarito. A resposta é sim para a versão final com o Gemma 4 E4B, cujas buscas devolveram as mesmas cartas que as do gabarito em 96,7% das consultas cujo gabarito contém cartas; com os outros modelos, o acerto foi menor, e os limites do CSW restringem o alcance da verificação (Seção 4.5.4).

4.6.4 Recomendação por Cenário

Recomenda-se, como configuração padrão, a versão final com o Gemma 4 E4B. Em GPU modesta, as opções são o controle com o Qwen 3 4B ou a versão final com o Gemma 4 E2B, e, quando só se admite uma chamada ao modelo, a configuração só com as descrições revisadas (Quadro 2). A escolha seguiu o critério da acurácia (Seção 3.3.6.2), e o *recall* menor da recusa e as cerca de duas chamadas ao modelo por consulta entram como custo declarado. A escolha do modelo com a versão final não passou por teste pareado; ela se apoia no desempate do conjunto de referência e na maior acurácia observada no conjunto de teste, e a versão final foi desenvolvida com o Gemma 4 E4B, o que pode favorecê-lo. As duas opções de GPU modesta tampouco foram comparadas por teste. Numa estação com mais VRAM, o candidato a avaliar é o GPT-OSS 20B, distribuído para o Ollama com cerca de 14 GB (OLLAMA, 2025a), medido só em nuvem e com a primeira versão.

Na implantação, as ferramentas, o validador e a leitura das chamadas escritas em texto acompanham a versão final e precisam ser atualizados junto com o catálogo, e convém fixar as versões do Ollama e do modelo e repetir o subconjunto manual a cada troca.

Quadro 2 – Configurações recomendadas por cenário de implantação

Cenário	**Configuração**	**Fundamento**	**Ressalva**
Padrão	versão final com o Gemma 4 E4B	maior acurácia no conjunto de teste; recusa com motivo ao usuário	versão desenvolvida com esse modelo; cerca de duas chamadas ao modelo; *recall* da recusa menor
Latência restrita	controle com o Gemma 4 E4B	uma chamada na maior parte das consultas	mais falsa recusa
GPU modesta (a)	controle com o Qwen 3 4B	maior acurácia observada entre os modelos menores	as opções (a) e (b) não foram comparadas por teste
GPU modesta (b)	versão final com o Gemma 4 E2B	menor latência na estação local	idem
Uma chamada ao modelo	só com as descrições revisadas, Gemma 4 E4B	sem laço; mesmo custo da primeira versão	sem ferramentas auxiliares nem retorno do validador

Fonte: Elaborado pelos autores, com os valores da Tabela 11 e da Seção 4.5.5.

Nota: Latências comparáveis só entre rodadas da mesma versão do Ollama (Seção 3.3.6.5).

4.6.5 Validade dos Resultados

As consultas são sintéticas, nenhuma foi formulada por um usuário real, e a proporção real de consultas fora do domínio é desconhecida. Uma mesma família de modelos (Claude) redigiu e anotou os conjuntos de validação e de teste e julgou os erros, embora em nenhum processo a IA tenha definido o gabarito, e a auditoria de 160 erros do conjunto de validação foi feita por um único agente, sem conferência humana. O código das ferramentas e do validador codifica o manual que define o gabarito, inclusive a tabela de datas. Os conjuntos de validação e de teste sortearam nomes e códigos das mesmas listas que as ferramentas consultam, e o alcance das ferramentas fora dessas listas não foi medido.

Nenhum conjunto público de avaliação foi empregado. Os de uso de ferramentas (Seção 2.3.4) estão, em geral, em inglês e não cobrem o catálogo do BDGEx, e os de *text-to-SQL*, como o Spider (YU et al., 2018), o BIRD (LI et al., 2023a) e o SQL-Eval (DEFOG, 2023), avaliam a geração da consulta SQL, que a solução não faz (Seção 3.2.1). Do SQL-Eval adotou-se o princípio da avaliação pela execução (Seção 4.5.4), e a avaliação num conjunto público fica como trabalho futuro (Seção 5.4).

A pontuação também tem vieses conhecidos. A primeira versão é penalizada por convenções do gabarito que não lhe foram dadas, das quais dependem 154 consultas do conjunto de validação, e nas consultas sem critério qualquer resposta que não busque conta como acerto. Cada rodada usou uma só data de execução, e os conjuntos de validação e de

teste tiveram uma execução por configuração. O pedido de esclarecimento foi respondido só pelo usuário simulado de resposta fixa, a versão final foi desenvolvida com um só modelo, e a extensão aos outros dois modelos usou outra versão do Ollama.

4.7 Entregáveis

Os códigos e os conjuntos de dados produzidos estão disponíveis no repositório público do projeto (CORRÊA; SANTANA, 2026), citado desde a Seção 3.2.1, que reúne os seguintes entregáveis:

- o *backend* em Python com *tool calling* nativo e as três versões da tradução, selecionáveis por configuração;
- as ferramentas do domínio e o validador;
- os três conjuntos de consultas, com o manual de anotação completo e a auditoria do gabarito;
- o *pipeline* de avaliação, com o teste A/B;
- a avaliação pela execução no acervo real;
- os registros de todas as rodadas, que permitem repontuar os resultados sem executar os modelos.

As listagens das consultas, a auditoria detalhada, a descrição dos ambientes e os resultados complementares, que não cabem no corpo do texto, também estão no repositório.

5 CONCLUSÃO

5.1 Retomada do Objetivo e Entrega

O objetivo geral do trabalho (Seção 1.2.1) foi adequar o projeto `prototipo_busca_llm`, do 1º CGEO, para empregar os recursos de *tool calling* na tradução de consultas em linguagem natural para critérios de busca no acervo cartográfico da DSG. O objetivo foi atingido, com as ressalvas registradas na Seção 4.6.1. O *backend* foi reescrito em Python com *tool calling* nativo, e a tradução evoluiu em três versões, até a versão final, com ferramentas auxiliares do domínio e verificação dos parâmetros com retorno ao modelo. Quanto ao efeito da adoção (Seção 4.6.3), a versão final obteve acurácia maior que a da primeira versão com os três modelos avaliados, o ganho do *tool calling* que se repetiu fora da amostra foi a recusa explícita das consultas fora do domínio, a vantagem em acurácia sobre a saída estruturada dependeu da versão da tradução e do modelo, e não houve ganho de tempo de processamento.

A entrega, descrita na Seção 4.7 e reunida no repositório público do projeto (CORRÊA; SANTANA, 2026), compreende o *backend* com as três versões da tradução, as ferramentas auxiliares e o validador, os conjuntos de consultas e o instrumental de avaliação, cujo alcance é discutido na Seção 5.2.

Para a DSG, o resultado é uma busca em linguagem natural executada em *hardware* próprio, com modelo de pesos abertos e sem serviço de nuvem, o que preserva o controle sobre os dados. O alcance dessa busca diante de pedidos reais está delimitado na Seção 5.3.

5.2 Contribuições

O trabalho apresenta quatro contribuições, que continuarão úteis mesmo quando os modelos avaliados forem substituídos:

1. As ferramentas auxiliares do domínio e o validador, que incorporam à tradução as convenções da cartografia sistemática (Seção 4.2). A identificação de nomes trata siglas, homônimos e a divisão político-administrativa. A normalização de códigos aplica a articulação das folhas pelo INOM e pelo MI, e a normalização de escalas, o enumerado de escalas do catálogo. A resolução de períodos converte expressões de tempo em datas a partir da data de execução, o pedido de esclarecimento trata a consulta sem critério, e a verificação dos parâmetros devolve ao modelo os erros encontrados antes da busca. Essas convenções, que o `prototipo_busca_llm` aplicava num dicionário fixo e que as duas primeiras versões informavam apenas nas des-

crições dos parâmetros, passaram a ser código verificável, consultado pelo modelo durante a resposta ou aplicado pelo validador.

2. Os três conjuntos de consultas e o manual de anotação que define o gabarito deles (Seção 3.3.3.2). O gabarito do conjunto de referência segue o manual e foi auditado por uma anotação independente e às cegas, com as divergências adjudicadas pelos autores, e o dos conjuntos de validação e de teste foi fixado antes da redação de cada consulta, a partir de valores reais dos metadados do BDGEx e da lista de municípios do IBGE (Seção 3.3.5).

3. O protocolo de comparação reprodutível, com testes pareados e o registro de cada rodada, que permite avaliar qualquer novo modelo ou nova configuração pelo mesmo critério. Como o resultado da mesma comparação mudou com o conjunto de consultas, a versão da tradução e o modelo, esse instrumento tende a permanecer útil por mais tempo que a classificação dos modelos avaliados, que se torna obsoleta à medida que novos modelos são lançados.

4. A avaliação pela execução no acervo real, que aplicou a interseção espacial entre as folhas e as malhas territoriais do IBGE e comparou as cartas devolvidas com as do gabarito (Seção 4.5.4), e o diagnóstico dos vícios do acervo coletado pelo CSW, como as folhas homônimas, a mesma folha em vários registros, as duas grafias do MI e os campos ausentes, de interesse direto para a DSG (Seção 4.1).

5.3 Limitações

As conclusões valem dentro dos limites discutidos na Seção 4.6.5, dos quais esta seção apenas nomeia os de maior peso. O primeiro é o caráter sintético das consultas, que deixa sem medida o desempenho diante de pedidos reais. O segundo é a circularidade entre o manual de anotação, as ferramentas e o gabarito, agravada pelo fato de uma mesma família de modelos de linguagem ter redigido, anotado e julgado os conjuntos de validação e de teste. O terceiro é o alcance da comparação com o controle, que não separa o mecanismo de *tool calling* das peças do domínio recebidas só pela versão final (Seção 3.3.6.4), somado ao desenvolvimento dessa versão com um único modelo (Seção 4.6.4).

O pedido de esclarecimento não foi exercitado com pessoas (Seção 4.2.3.6), e a avaliação pela execução no acervo real ficou restrita aos metadados disponíveis no CSW (Seção 4.5.4).

5.4 Trabalhos Futuros

Os trabalhos futuros decorrem das limitações, dos erros restantes da versão final e dos temas deixados fora do escopo:

1. implementar predicados espaciais relacionais, como “perto de”, “num raio de” e “ao norte de”, com a geocodificação do lugar citado, novos parâmetros na ferramenta de busca e operadores de distância e de direção (Seção 4.2.4);

2. desambiguar folhas e municípios homônimos pela UF ou pelo código, tornar a busca insensível a acentos e ao zero à esquerda do MI e agrupar as edições e as duplicatas do catálogo;

3. avaliar o pedido de esclarecimento com usuários reais, em conversas de vários turnos;

4. suprir as lacunas do validador e da recusa contestada, como a palavra-chave preenchida com região, projeto ou escala, as expressões de tempo fora da tabela do manual e a contestação de recusas corretas, e medir o efeito em um terceiro conjunto de consultas, pois o conjunto de teste, já empregado na avaliação, deixou de ser inédito;

5. coletar consultas reais anotadas por especialistas e avaliar a solução com usuários do catálogo, em comparação com a busca atual por texto simples (Tabela 1), como previam as demandas do 1º CGEO, com interface e integração ao sistema da DSG (autenticação, controle de acesso e monitoramento);

6. avaliar os modelos empregados num conjunto público de chamada de ferramentas, como o BFCL (PATIL et al., 2025), com um subconjunto de consultas traduzido para o português, de modo a situar o desempenho deles fora do domínio do catálogo, comparação que não foi feita porque os conjuntos públicos disponíveis estão em inglês e não cobrem o BDGEx (Seção 4.6.5);

7. coletar o catálogo completo, com o projeto, as datas de publicação e de criação e as áreas de suprimento oficiais;

8. avaliar localmente modelos maiores, como o GPT-OSS 20B (OPENAI, 2025), distribuído para o Ollama com cerca de 14 GB (OLLAMA, 2025a) e compatível com uma GPU de 16 GB, e o Qwen 3.8 27B (QWEN TEAM, 2026), que ocupa 18 GB no Ollama e exige GPU com mais memória, e medir na estação local, com a GPU livre de outros programas, as duas opções recomendadas para GPU modesta;

9. construir uma versão leve para testes e demonstrações, com o banco num arquivo local, como o SpatiaLite, extensão espacial do SQLite, ou o DuckDB com a sua extensão espacial, e com os modelos executados diretamente pela biblioteca Transfor-

mers, do Hugging Face, de modo que a solução dispense servidor de banco, servidor de modelos e contêineres;

10. avaliar o modo de raciocínio dos modelos e o uso de exemplos resolvidos no *prompt*, com a medida do ganho de acurácia e do custo em latência, e investigar por que o controle obteve acurácia maior que a da versão final com o Qwen 3 4B;

11. criar um mecanismo de contingência que, esgotado o limite do laço de chamadas sem busca válida, encaminhe a consulta à busca textual do banco e registre a falha, para que ela não fique encoberta como no `prototipo_busca_llm` (Seção 2.2.2);

12. treinar, por ajuste fino, um modelo menor para o domínio, tema excluído do escopo deste trabalho (Seção 1.1).

REFERÊNCIAS

1º CENTRO DE GEOINFORMAÇÃO. *Protótipo de busca de geoinformação utilizando Large Language Models*. 2025. Repositório GitHub. Disponível em: <https://github.com/1cgeo/prototipo_busca_llm>. Acesso em: mar. 2026.

ABDIN, M.; ANEJA, J.; BEHL, H.; BUBECK, S.; ELDAN, R.; GUNASEKAR, S.; HARRISON, M.; HEWETT, R. J.; JAVAHERIPI, M.; KAUFFMANN, P. et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024. Disponível em: <https://arxiv.org/abs/2412.08905>.

ARTSTEIN, R.; POESIO, M. Inter-coder agreement for computational linguistics. *Computational Linguistics*, v. 34, n. 4, p. 555–596, 2008.

BRASIL. *Decreto-Lei n.º 243, de 28 de fevereiro de 1967: Fixa as Diretrizes e Bases da Cartografia Brasileira e dá outras providências*. Brasília, DF, 1967. Institui o Sistema Cartográfico Nacional e as escalas-padrão da Cartografia Sistemática Terrestre Básica. Disponível em: <https://www.planalto.gov.br/ccivil_03/decreto-lei/1965-1988/del0243.htm>. Acesso em: set. 2026.

BRASIL. *Decreto n.º 6.666, de 27 de novembro de 2008: Institui, no âmbito do Poder Executivo federal, a Infra-Estrutura Nacional de Dados Espaciais – INDE, e dá outras providências*. Brasília, DF, 2008. Publicado no Diário Oficial da União de 28 nov. 2008; redação alterada pelo Decreto n.º 12.402, de 13 de março de 2025. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2008/decreto/d6666.htm>. Acesso em: set. 2026.

BRASIL. Comissão Nacional de Cartografia (CONCAR). *Perfil de Metadados Geoespaciais do Brasil (Perfil MGB)*. [S.l.], 2009. Versão homologada em novembro de 2009; atualizada pelo Perfil MGB 2.0 (IBGE; DSG, 2021). Disponível em: <https://inde.gov.br/pdf/Perfil_MGB_homologado_nov2009_v1.pdf>. Acesso em: set. 2026.

BRASIL. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico. *Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG)*. [S.l.], 2016. Norma EB80-N-72.001, 2.ª edição, aprovada pela Portaria n.º 008-DCT, de 10 de fevereiro de 2016. Disponível em: <https://geoportal.eb.mil.br/portal/index.php/normas/et-pcdg>. Acesso em: set. 2026.

BROWN, T.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J. D.; DHARIWAL, P.; NEELAKANTAN, A.; SHYAM, P.; SASTRY, G.; ASKELL, A. et al. Language models are few-shot learners. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2020. v. 33, p. 1877–1901. Disponível em: <https://arxiv.org/abs/2005.14165>.

CHASE, H. *LangChain: Building applications with LLMs through composability*. 2022. Disponível em: <https://github.com/langchain-ai/langchain>. Acesso em: mar. 2026.

CHEN, X.; LIN, M.; SCHÄRLI, N.; ZHOU, D. Teaching large language models to self-debug. In: *International Conference on Learning Representations (ICLR)*. [s.n.],

2024. p. 8746–8825. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html>.

COHEN, J. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, v. 20, n. 1, p. 37–46, 1960.

COLVIN, S. *Pydantic: Data validation using Python type hints*. 2019. Disponível em: <https://docs.pydantic.dev>. Acesso em: mar. 2026.

CORRÊA, V. H.; SANTANA, D. *Busca de geoinformação em linguagem natural com tool calling: código, conjuntos de avaliação e resultados*. 2026. Repositório GitHub. Disponível em: <https://github.com/vhenequim/bdgex-busca-tool-calling>. Acesso em: 12 out. 2026.

DEFOG. *SQL-Eval: evaluate the accuracy of LLM generated outputs*. 2023. Repositório GitHub, licença Apache 2.0. Compara os resultados da execução da consulta gerada e da consulta de referência. Disponível em: <https://github.com/defog-ai/sql-eval>. Acesso em: 12 out. 2026.

DIETTERICH, T. G. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, v. 10, n. 7, p. 1895–1923, 1998.

DIRETORIA DE SERVIÇO GEOGRÁFICO DO EXÉRCITO BRASILEIRO. *Acesso ao BDGEx*. 2025a. Geoportal do Exército. Disponível em: <https://geoportal.eb.mil.br/portal/index.php/bdgex/acesso-ao-bdgex>. Acesso em: set. 2026.

DIRETORIA DE SERVIÇO GEOGRÁFICO DO EXÉRCITO BRASILEIRO. *Missão da Diretoria de Serviço Geográfico*. 2025b. Geoportal do Exército. Disponível em: <https://geoportal.eb.mil.br/portal/index.php/chefia/missao>. Acesso em: set. 2026.

DWORK, C.; FELDMAN, V.; HARDT, M.; PITASSI, T.; REINGOLD, O.; ROTH, A. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, v. 349, n. 6248, p. 636–638, 2015. Disponível em: <https://doi.org/10.1126/science.aaa9375>.

EFRON, B.; TIBSHIRANI, R. J. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

FARABET, C.; LACOMBE, O. *Gemma 4: byte for byte, the most capable open models*. 2026. Anúncio do Gemma 4, 2 abr. 2026; licença Apache 2.0. Disponível em: <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>. Acesso em: out. 2026.

FRANTAR, E.; ASHKBOOS, S.; HOEFLER, T.; ALISTARH, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2023. Disponível em: <https://arxiv.org/abs/2210.17323>.

GEMMA TEAM. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026. Google DeepMind. Disponível em: <https://arxiv.org/abs/2607.02770>.

GENG, S.; COOPER, H.; MOSKAL, M.; JENKINS, S.; BERMAN, J.; RANCHIN, N.; WEST, R.; HORVITZ, E.; NORI, H. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. *arXiv preprint arXiv:2501.10868*, 2025. As

versões 1 e 2 do *preprint* têm o título "Generating structured outputs from language models: benchmark and studies". Disponível em: <https://arxiv.org/abs/2501.10868>.

GGML CONTRIBUTORS. *GGUF: file format specification*. 2023. Formato introduzido no `llama.cpp` em agosto de 2023. Disponível em: <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>. Acesso em: set. 2026.

GOOGLE DEEPMIND. *Gemma 4 QAT models: optimizing model compression for mobile and laptop efficiency*. 2026. Checkpoints com quantização consciente de treinamento (Q4_0), 5 jun. 2026. Disponível em: <https://www.blog.google/innovation-and-ai/technology/developers-tools/quantization-aware-training-gemma-4/>. Acesso em: out. 2026.

GOU, Z.; SHAO, Z.; GONG, Y.; SHEN, Y.; YANG, Y.; DUAN, N.; CHEN, W. CRITIC: Large language models can self-correct with tool-interactive critiquing. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 57734–57811. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/fef126561bbf9d4467dbb8d27334b8fe-Abstract-Conference.html>.

GROQ. *Reasoning*. 2026. Documentação do parâmetro `reasoning_effort`. Disponível em: <https://console.groq.com/docs/reasoning>. Acesso em: set. 2026.

HOLM, S. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, v. 6, n. 2, p. 65–70, 1979.

HU, Y.; JANOWICZ, K.; PRASAD, S.; GAO, S. Metadata topic harmonization and semantic search for linked-data-driven geoportals: A case study using ArcGIS Online. *Transactions in GIS*, v. 19, n. 3, p. 398–416, 2015. Disponível em: <https://doi.org/10.1111/tgis.12151>.

HUANG, J.; CHEN, X.; MISHRA, S.; ZHENG, H. S.; YU, A. W.; SONG, X.; ZHOU, D. Large language models cannot self-correct reasoning yet. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 32808–32824. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/8b4add8b0aa8749d80a34ca5d941c355-Abstract-Conference.html>.

HUANG, S.; FANG, Z.; CHEN, Z.; YUAN, S.; YE, J.; ZENG, Y.; CHEN, L.; MAO, Q.; ZHAO, F. CRITICTOOL: Evaluating self-critique capabilities of large language models in tool-calling error scenarios. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, 2025. p. 26672–26704. Disponível em: <https://aclanthology.org/2025.emnlp-main.1355/>.

IBGE. *Noções básicas de cartografia*. Rio de Janeiro: Instituto Brasileiro de Geografia e Estatística, 1999. (Manuais técnicos em geociências, 8). ISBN 85-240-0751-6. Disponível em: <https://biblioteca.ibge.gov.br/visualizacao/livros/liv8595_v1.pdf>. Acesso em: out. 2026.

KHATTAB, O.; SINGHVI, A.; MAHESHWARI, P.; ZHANG, Z.; SANTHANAM, K.; A, S. V.; HAQ, S.; SHARMA, A.; JOSHI, T. T.; MOAZAM, H. et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p.

54928–54958. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html>.

LANGCHAIN AI. *LangGraph: Building stateful, multi-actor applications with LLMs*. 2024. Disponível em: <https://github.com/langchain-ai/langgraph>. Acesso em: mar. 2026.

LI, H.; YUE, P.; WU, H.; TENG, B.; ZHAO, Y.; LIU, C. A question-answering framework for geospatial data retrieval enhanced by a knowledge graph and large language models. *International Journal of Digital Earth*, v. 18, n. 1, 2025. Art. 2510566. Disponível em: <https://doi.org/10.1080/17538947.2025.2510566>.

LI, J.; HUI, B.; QU, G.; YANG, J.; LI, B.; LI, B.; WANG, B.; QIN, B.; GENG, R.; HUO, N. et al. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 42330–42357. Datasets and Benchmarks Track. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html>.

LI, M.; ZHAO, Y.; YU, B.; SONG, F.; LI, H.; YU, H.; LI, Z.; HUANG, F.; LI, Y. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023. p. 3102–3116. Disponível em: <https://aclanthology.org/2023.emnlp-main.187/>.

LI, Z.; NING, H. Autonomous GIS: the next-generation AI-powered GIS. *International Journal of Digital Earth*, v. 16, n. 2, p. 4668–4686, 2023. Disponível em: <https://doi.org/10.1080/17538947.2023.2278895>.

LIU, J. *Instructor: Structured outputs for LLMs*. 2023. Disponível em: <https://python.useinstructor.com>. Acesso em: mar. 2026.

LIU, X.; SHEN, S.; LI, B.; MA, P.; JIANG, R.; ZHANG, Y.; FAN, J.; LI, G.; TANG, N.; LUO, Y. A survey of text-to-SQL in the era of LLMs: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering*, v. 37, n. 10, p. 5735–5754, 2025. Disponível em: <https://ieeexplore.ieee.org/document/11095853>.

LU, J.; HOLLEIS, T.; ZHANG, Y.; AUMAYER, B.; NAN, F.; BAI, H.; MA, S.; MA, S.; LI, M.; YIN, G. et al. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. Albuquerque, New Mexico: Association for Computational Linguistics, 2025. p. 1160–1183. Disponível em: <https://aclanthology.org/2025.findings-naacl.65/>.

MADAAN, A.; TANDON, N.; GUPTA, P.; HALLINAN, S.; GAO, L.; WIEGREFFE, S.; ALON, U.; DZIRI, N.; PRABHUMOYE, S.; YANG, Y. et al. Self-Refine: Iterative refinement with self-feedback. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 46534–46594. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html>.

MAI, G.; HUANG, W.; SUN, J.; SONG, S.; MISHRA, D.; LIU, N.; GAO, S.; LIU, T.; CONG, G.; HU, Y. et al. On the opportunities and challenges of foundation models for GeoAI (vision paper). *ACM Transactions on Spatial Algorithms and Systems*, v. 10, n. 2, p. 1–46, 2024. Disponível em: <https://doi.org/10.1145/3653070>.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Disponível em: <https://nlp.stanford.edu/IR-book/>.

MANVI, R.; KHANNA, S.; MAI, G.; BURKE, M.; LOBELL, D.; ERMON, S. GeoLLM: Extracting geospatial knowledge from large language models. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 38791–38807. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/a87f2df7c4ab0213c6ea228e7b7f0a4d-Abstract-Conference.html>.

MCNEMAR, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, v. 12, n. 2, p. 153–157, 1947.

MICROSOFT RESEARCH. *Phi-4*. 2024. Model card no Hugging Face; lançado em 12 dez. 2024; licença MIT. Disponível em: <https://huggingface.co/microsoft/phi-4>. Acesso em: out. 2026.

MISTRAL AI. *Mistral NeMo*. 2024. Disponível em: <https://mistral.ai/news/mistral-nemo>. Acesso em: set. 2026.

NING, H.; LI, Z.; AKINBOYEWA, T.; LESSANI, M. N. An autonomous GIS agent framework for geospatial data retrieval. *International Journal of Digital Earth*, v. 18, n. 1, 2025. Art. 2458688. Disponível em: <https://doi.org/10.1080/17538947.2025.2458688>.

OBJECT MANAGEMENT GROUP. *Business Process Model and Notation (BPMN), version 2.0.2*. 2014. OMG Document Number formal/13-12-09. Disponível em: <https://www.omg.org/spec/BPMN/2.0.2/PDF>. Acesso em: 12 out. 2026.

OLLAMA. *Ollama: Get up and running with large language models locally*. 2023. Disponível em: <https://ollama.com>. Acesso em: mar. 2026.

OLLAMA. *mistral-nemo:12b*. 2024. Biblioteca de modelos do Ollama. Disponível em: <https://ollama.com/library/mistral-nemo:12b>. Acesso em: out. 2026.

OLLAMA. *gpt-oss:20b*. 2025. Biblioteca de modelos do Ollama. Disponível em: <https://ollama.com/library/gpt-oss:20b>. Acesso em: set. 2026.

OLLAMA. *qwen3:4b-instruct-2507-q4_K_M*. 2025. Biblioteca de modelos do Ollama. Disponível em: <https://ollama.com/library/qwen3:4b-instruct-2507-q4_K_M>. Acesso em: out. 2026.

OLLAMA. *gemma4*. 2026. Biblioteca de modelos do Ollama; tags `gemma4:e4b-it-qat` (6,1 GB) e `gemma4:e2b-it-qat` (4,3 GB). Disponível em: <https://ollama.com/library/gemma4/tags>. Acesso em: out. 2026.

OPEN GEOSPATIAL CONSORTIUM. *OpenGIS Catalogue Services Specification, version 2.0.2 (Corrigendum 2 Release)*. 2007. OGC 07-006r1. Editores: Douglas Nebert, Arliss Whiteside e Panagiotis Vretanos. Disponível em: <https://docs.ogc.org/is/07-006r1/07-006r1.pdf>. Acesso em: set. 2026.

OPENAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025. Disponível em: <https://arxiv.org/abs/2508.10925>.

OUYANG, L.; WU, J.; JIANG, X.; ALMEIDA, D.; WAINWRIGHT, C.; MISHKIN, P.; ZHANG, C.; AGARWAL, S.; SLAMA, K.; RAY, A. et al. Training language models to follow instructions with human feedback. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2022. v. 35, p. 27730–27744. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html>.

PATIL, S. G.; MAO, H.; YAN, F.; JI, C. C.-J.; SURESH, V.; STOICA, I.; GONZALEZ, J. E. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In: *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 2025. (Proceedings of Machine Learning Research, v. 267), p. 48371–48392. Disponível em: <https://proceedings.mlr.press/v267/patil25a.html>.

PATIL, S. G.; ZHANG, T.; WANG, X.; GONZALEZ, J. E. Gorilla: Large language model connected with massive APIs. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2024. v. 37, p. 126544–126565. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html>.

PEDRO, R.; COIMBRA, M. E.; CASTRO, D.; CARREIRA, P.; SANTOS, N. Prompt-to-SQL injections in LLM-integrated web applications: risks and defenses. In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Ottawa, Canada: IEEE, 2025. p. 1768–1780. Versão anterior em arXiv:2308.01990, com o título "From prompt injections to SQL injection attacks: how protected is your LLM-integrated web application?". Disponível em: <https://doi.org/10.1109/ICSE55347.2025.00007>.

POSTGIS DEVELOPMENT TEAM. *PostGIS: Spatial and geographic objects for PostgreSQL*. 2001. Disponível em: <https://postgis.net>. Acesso em: mar. 2026.

QIN, Y.; LIANG, S.; YE, Y.; ZHU, K.; YAN, L.; LU, Y.; LIN, Y.; CONG, X.; TANG, X.; QIAN, B. et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 9695–9717. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html>.

QWEN TEAM. *Qwen3-4B-Instruct-2507*. 2025. Model card no Hugging Face (Alibaba Cloud); lançado em 6 ago. 2025; pesos abertos sob licença Apache 2.0. Disponível em: <https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>. Acesso em: out. 2026.

QWEN TEAM. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. Disponível em: <https://arxiv.org/abs/2505.09388>.

QWEN TEAM. *Qwen3.8-27B*. 2026. Model card no Hugging Face (Alibaba Cloud); pesos abertos sob licença Apache 2.0. Disponível em: <https://huggingface.co/Qwen/Qwen3.8-27B>. Acesso em: set. 2026.

RAMÍREZ, S. *FastAPI: Modern, fast web framework for building APIs with Python*. 2018. Disponível em: <https://fastapi.tiangolo.com>. Acesso em: mar. 2026.

ROSS, H.; MAHABALESHWARKAR, A. S.; SUHARA, Y. When2Call: When (not) to call tools. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Albuquerque, New Mexico: Association for Computational Linguistics, 2025. p. 3391–3409. Disponível em: <https://aclanthology.org/2025.naacl-long.174/>.

SCHICK, T.; DWIVEDI-YU, J.; DESSÌ, R.; RAILEANU, R.; LOMELI, M.; HAMBRO, E.; ZETTLEMOYER, L.; CANCEDDA, N.; SCIALOM, T. Toolformer: Language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 68539–68551. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html>.

SHINN, N.; CASSANO, F.; GOPINATH, A.; NARASIMHAN, K.; YAO, S. Reflexion: Language agents with verbal reinforcement learning. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 8634–8652. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html>.

TAM, Z. R.; WU, C.-K.; TSAI, Y.-L.; LIN, C.-Y.; LEE, H.-y.; CHEN, Y.-N. Let me speak freely? A study on the impact of format restrictions on large language model performance. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Miami, Florida, US: Association for Computational Linguistics, 2024. p. 1218–1236. Disponível em: <https://aclanthology.org/2024.emnlp-industry.91/>.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017. v. 30. Disponível em: <https://arxiv.org/abs/1706.03762>.

WEI, J.; TAY, Y.; BOMMASANI, R.; RAFFEL, C.; ZOPH, B.; BORGEAUD, S.; YOGATAMA, D.; BOSMA, M.; ZHOU, D.; METZLER, D. et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. Disponível em: <https://arxiv.org/abs/2206.07682>.

WILSON, E. B. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, v. 22, n. 158, p. 209–212, 1927.

XAVIER, E. M. A.; MEYER, W. S.; LUNARDI, O. A. Banco de dados geográficos do Exército Brasileiro: arquitetura e resultados. In: *Jornadas Ibéricas de Infraestruturas de Dados Espaciais (JIIDE 2014)*. [s.n.], 2014. Disponível em: <https://www.idee.es/resources/presentaciones/JIIDE14/20141106/BancoDadosGeograficos_presentacion.pdf>. Acesso em: set. 2026.

YANG, C.; WANG, X.; LU, Y.; LIU, H.; LE, Q. V.; ZHOU, D.; CHEN, X. Large language models as optimizers. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 12028–12068. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/3339f19c5fcee3ad74502947a32be9e6-Abstract-Conference.html>.

YAO, S.; SHINN, N.; RAZAVI, P.; NARASIMHAN, K. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In: *International*

Conference on Learning Representations (ICLR). [s.n.], 2025. p. 9965–10017. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html>.

YAO, S.; ZHAO, J.; YU, D.; DU, N.; SHAFRAN, I.; NARASIMHAN, K.; CAO, Y. ReAct: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2023. Disponível em: <https://arxiv.org/abs/2210.03629>.

YU, T.; ZHANG, R.; YANG, K.; YASUNAGA, M.; WANG, D.; LI, Z.; MA, J.; LI, I.; YAO, Q.; ROMAN, S. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018. p. 3911–3921. Disponível em: <https://aclanthology.org/D18-1425/>.

ZHANG, Y.; WEI, C.; HE, Z.; YU, W. GeoGPT: An assistant for understanding and processing geospatial tasks. *International Journal of Applied Earth Observation and Geoinformation*, v. 131, 2024. Art. 103976. Disponível em: <https://doi.org/10.1016/j.jag.2024.103976>.

ZHAO, A.; HUANG, D.; XU, Q.; LIN, M.; LIU, Y.-J.; HUANG, G. ExpeL: LLM agents are experiential learners. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Association for the Advancement of Artificial Intelligence, 2024. v. 38, n. 17, p. 19632–19642. Disponível em: <https://ojs.aaai.org/index.php/AAAI/article/view/29936>.